Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

DPD Parcel Wizard

Project Engineering

Year 4

Anthony Johnson

Bachelor of Engineering (Honours) in Software and Electronic Engineering

Atlantic Technological University

2025/2026

Declaration

This project is presented in partial fulfilment of the requirements for the degree of Bachelor of Engineering (Honours) in Software and Electronic Engineering at Atlantic Technological University.

This project is my own work, except where otherwise accredited. Where the work of others has been used or incorporated during this project, this is acknowledged and referenced.

_____

Acknowledgements

I would like to thank David Newell, my project supervisor in ATU, for his guidance and support throughout the year. His advice helped me stay focused, organised, and accountable as the project developed from an initial idea into a working prototype.

I would also like to thank Gail Wood, Knowledge and Collaboration Manager in DPD and my former supervisor during my placement with the IT team. Gail suggested the original idea of developing a Parcel Wizard app and provided helpful feedback at different stages of the project. One of the most important pieces of feedback was the suggestion to include a Manage Delivery section, which became a key feature of the final application.

Finally, I would like to thank DPD for giving me the opportunity to apply my software engineering skills in a real-world business context. This project allowed me to work on a practical logistics problem connected to an organisation I know well, making the experience both valuable and meaningful.

Table of Contents

1 Summary 10
2 Poster 11
3 Introduction 12
3.1 Disclaimer (Page Count) 12
4 Project Context and Background 13
4.1 Personal Background and Experience with DPD 13
4.2 Motivation and Opportunity 13
4.3 Learning React and React Native 13
4.4 Parcel Wizard Service 14
4.5 Research into the Existing Parcel Wizard Website and App 14
4.6 Competitor Research 16
5 Project Architecture 17
6 Project Plan 18
7 Technology Stack and Architecture 19
7.1 React-Native and Expo 19
7.2 Navigation using Expo Router 19
7.2.1 Navigation Structure 19
7.2.2 Root Stack Navigation 20
7.2.3 Drawer Navigation 20
7.2.4 Bottom Tab Navigation 21
7.3 State Management with React Context 22
7.3.1 Context Architecture 22
7.3.2 AuthContext 22

7.3.3 Account Context 24
7.3.4 Parcel Context 27
7.3.5 Context Summary 28
8 Authentication and Secure Storage 29
8.1 Login Flow 29
8.1.1 Login Screen UI 31
8.1.2 Login Service Layer 32
8.1.3 Post-Login Processing 33
8.2 Registration Flow 33
8.3 Dev Login / Mock Access 34
8.4 Navigation After Login 34
8.5 Secure Storage of Credentials 35
8.6 Login Summary 35
9 Account Management 35
9.1 Account Tab Screen 36
9.1.1 Account Dropdown Component 36
9.1.2 Account Details Panel 37
9.1.3 Mobile Number Management 37
9.1.4 Email Address Management 39
9.2 Address Book 40
9.2.1 Address Creation and Editing 41
9.2.2 Default Address Protection 41
9.2.3 Rendering Address Cards 42
9.3 Account Management Summary 43

10 Home Screen and Quick Actions 44
10.1 Home Screen Logic 44
10.2 Welcome Card 45
10.3 Quick Actions 45
10.4 Track 46
10.4.1 Home Screen Integration 46
10.4.2 Slide Down Panel 47
10.4.3 Tracking Box 49
10.4.4 Submission Flow 50
10.5 Scan 50
10.5.1 Scan Screen Structure 50
10.5.2 Scan Behaviour 51
10.6 Locate 52
10.6.1 Locate Hook 52
10.6.2 Map Screen 53
10.7 Support 54
10.7.1 Support Screen 54
10.7.2 FAQs 54
10.8 Delivery Feed 57
10.8.1 Arriving Today Section 57
10.8.2 Recently Delivered Section 57
10.8.3 Parcel Card 58
10.9 Home Screen Summary 58
11 Parcel Details 59

11.1 Fetching Consignment Data 59
11.1.1 Loading and Empty States 59
11.2 Parcel Detail View and Tracking History 60
11.2.1 Main Screen Composition 60
11.2.2 Status Summary 60
11.2.3 Parcel Information Section 61
11.2.4 Proof of Delivery 61
11.2.5 Tracking History 62
11.2.6 Parcel Support 64
11.3 Parcel Details Summary 64
12 Calendar and Scheduling 65
12.1 Main Calendar Screen 65
12.1.1 Calendar Interaction Flow 66
12.1.2 Rendering selected-day parcels 67
12.2 Calendar Screen Hook 67
12.2.1 Visible Month and Selected Date 68
12.2.2 Delivery Date Set 68
12.2.3 Selected Date Parcels 68
12.2.4 Month Navigation 69
12.3 Calendar Container Component 69
12.3.1 Month Header and Navigation 70
12.3.2 Weekday Row 71
12.3.3 Day Grid and Day Cells 71
12.4 Schedule Creation 73

12.4.1 Create Delivery Schedule Button 74
12.4.2 Date Selection Mode 74
12.4.3 Continue to Schedule Details 75
12.4.4 Schedule Details Card 76
12.4.5 Delivery Actions 77
12.4.6 Form Validation 79
12.4.7 Schedule Payload 80
12.4.8 Schedule Map and Calendar Markers 81
12.5 Calendar Legend 82
12.6 Calendar Summary 83
13 Manage Delivery and Delivery History 84
13.1 Main Screen Logic 84
13.1.1 Incoming and History List Preparation 84
13.2 Tab Switching 85
13.3 Incoming Parcels View 85
13.3.1 Incoming Parcel Card 86
13.3.2 Manage Availability 87
13.4 Manage Delivery Form 87
13.4.1 Address Selection 88
13.4.2 Validation inside the Form 89
13.4.3 Form Submission Flow 90
13.5 History View 91
13.5.1 History Parcel Card 92
13.6 Empty State 92

13.7 Manage Delivery and Delivery History Summary 92
14 Utilities and Helpers 93
14.1 Date Utilities 93
14.1.1 Calendar Day Type 94
14.1.2 isSameDay() 94
14.1.3 startOfMonth() and addMonths() 94
14.1.4 formatDateKey() 95
14.1.5 buildCalendarDays() 95
14.2 Parcel Status Utilities 97
14.2.1 getParcelStatus() 97
14.2.2 getParcelStatusDisplay() 97
14.3 Mapping Helpers 98
14.3.1 mapConsignmentToParcel() 98
14.3.2 mapConsignmentsToParcel() 98
15 Ethics 100
16 Conclusion 101
17 Appendix 102
17.1 Github Link 102
17.2 Short Demo Video 102
17.3 Longer Demo Video 102
18 References 103

1 Summary

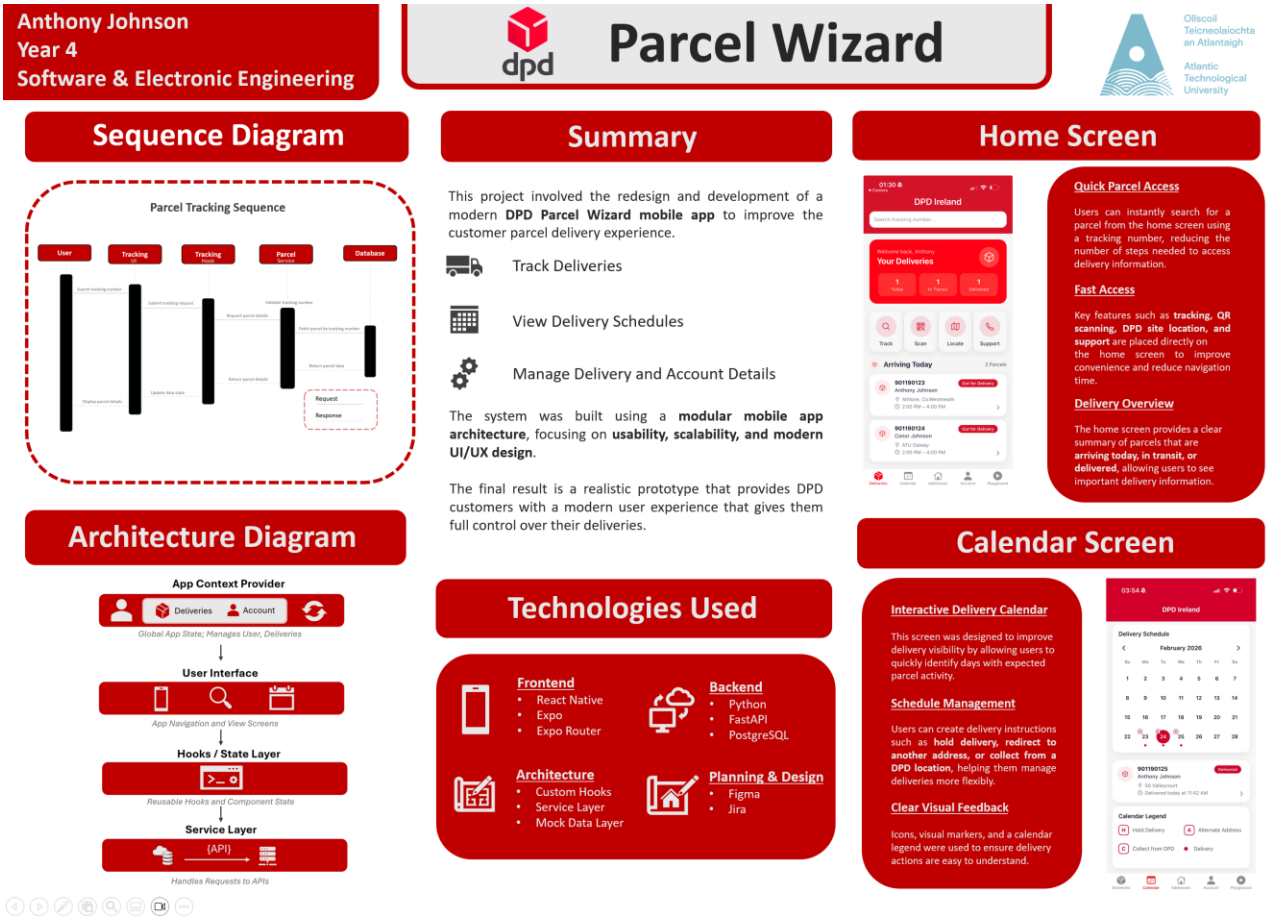
The goal of this project was to design and develop a modern mobile application for DPD Parcel Wizard. The original motivation behind the project came from a suggestion by Paul Lennon, my CICD lecturer, to contact DPD and ask them if I could do anything for them. DPD then suggested reworking the Parcel Wizard app. Parcel Wizard already provides useful delivery management services, but the aim of this project was to reimagine those features in a cleaner, more accessible, and more modern mobile app.

The scope of the project focused on building a working mobile prototype using React Native and Expo. The application includes user authentication, account management, address book, parcel tracking, parcel details, proof of delivery display, delivery history, support features, a delivery calendar, and delivery scheduling. The app also includes quick actions from the home screen, allowing users to track, scan, locate, or contact support with minimal navigation.

The project was developed using a component-based approach. React Native was used for the mobile interface, Expo supported development and device features, and Expo Router was used for navigation. React Context was used to manage shared authentication, account, and parcel data across the application. The app was designed with separation of concerns in mind, using reusable components, hooks, service layers, mock data, and mapping helpers so that future backend integration would be easier.

The final outcome was a functional mobile app prototype that demonstrates the main Parcel Wizard customer features in a clear and user-friendly way. Overall, the project showed how a modern mobile interface could improve parcel visibility, customer control, and ease of use, while also giving me the opportunity to apply software engineering skills to a real-world logistics environment.

2 Poster


Anthony Johnson
Year 4
Software & Electronic Engineering
dpd
Parcel Wizard
Ollscoil Teicneolaíochta an Atlantaigh
Atlantic Technological University
Sequence Diagram
Parcel Tracking Sequence
User
Tracking UI
Tracking Svcs
Parcel Service
Database
Request
Response
Summary
This project involved the redesign and development of a modern DPD Parcel Wizard mobile app to improve the customer parcel delivery experience.
Track Deliveries
View Delivery Schedules
Manage Delivery and Account Details
The system was built using a modular mobile app architecture, focusing on usability, scalability, and modern UI/UX design.
The final result is a realistic prototype that provides DPD customers with a modern user experience that gives them full control over their deliveries.
Home Screen
DPD Ireland
Your Deliveries
Track
Scan
Locate
Support
Arriving Today
Quick Parcel Access
Users can instantly search for a parcel from the home screen using a tracking number, reducing the number of steps needed to access delivery information.
Fast Access
Key features such as tracking, QR scanning, DPD site location, and support are placed directly on the home screen to improve convenience and reduce navigation time.
Delivery Overview
The home screen provides a clear summary of parcels that are arriving today, in transit, or delivered, allowing users to see important delivery information.
Architecture Diagram
App Context Provider
Deliveries
Account
Global App State; Manages User, Deliveries
User Interface
App Navigation and View Screens
Hooks / State Layer
Reusable Hooks and Component State
Service Layer
{API}
Handles Requests to APIs
Technologies Used
Frontend
React Native
Expo
Expo Router
Backend
Python
FastAPI
PostgreSQL
Architecture
Custom Hooks
Service Layer
Mock Data Layer
Planning & Design
Figma
Jira
Calendar Screen
Interactive Delivery Calendar
This screen was designed to improve delivery visibility by allowing users to quickly identify days with expected parcel activity.
Schedule Management
Users can create delivery instructions such as hold delivery, redirect to another address, or collect from a DPD location, helping them manage deliveries more flexibly.
Clear Visual Feedback
Icons, visual markers, and a calendar legend were used to ensure delivery actions are easy to understand.
DPD Ireland
Delivery Schedule
February 2026
Calendar Legend

3 Introduction

The goal of this project was to design and develop a modern mobile application to replace the existing Parcel Wizard app. The original application presented several limitations in terms of usability, design, and overall user experience. As DPD is competing in a competitive logistics market, there is a clear need for an intuitive, responsive, and feature-rich mobile solution that meets modern user expectations.

The primary objective with this project was to improve how customers interact with their deliveries by creating a streamlined and user-friendly interface. The application focuses on providing users with full visibility and control over their parcels, including real-time tracking, delivery management options, and access to account information. Emphasis was placed on delivering a clean and engaging user experience, ensuring that common tasks such as tracking a parcel or updating delivery preferences can be completed quickly and efficiently.

In terms of scope, the project involved the design and implementation of a cross-platform mobile application using modern development tools and frameworks. Core features include user authentication, parcel tracking, delivery management, address and contact management, calendar-based scheduling, and support functionality. The project also considers integration with backend services and follows a modular architecture to allow for future scalability and enhancement.

3.1 Disclaimer (Page Count)

Although the report is quite long in terms of page count, this is mainly because the project is a mobile app. Many of the app features are visual and user-interface based, so screenshots are necessary to show screen layouts, navigation flow, forms, cards, modals, and user interactions. The screenshots are included to make the report clearer and to demonstrate the completed application in a way that text alone would not fully capture.

4 Project Context and Background

This section introduces the background and context behind the project. It explains my personal connection to DPD, the business problem that motivated the project, the research carried out into the existing Parcel Wizard service, and the technical learning required to develop the application. This background is important because the project was not only a software development exercise, but also an opportunity to apply technical skills to a real logistics environment that I already had experience with.

4.1 Personal Background and Experience with DPD

I have worked with DPD for six years, with five of those years spent working in the warehouse and one year completed as part of my college placement with the IT team. This gave me a strong understanding of both the operational and technical sides of the business.

Working in the warehouse helped me understand how parcels move through the network. My placement with the IT team then gave me a better understanding of how software systems in the background support those day-to-day operations. This combination of both practical and technical experience gave me a strong starting point for the Parcel Wizard app.

4.2 Motivation and Opportunity

Because of this background, I was excited to be given the opportunity to develop the Parcel Wizard mobile app. It felt like a real-world project rather than a purely academic one, because it was connected to a business and workflow that I already understood.

The project gave me the chance to apply the software development skills I have learned in college to a practical logistics problem. It also gave me the opportunity to create something that could be useful in a real customer-facing environment

4.3 Learning React and React Native

At the beginning of the project, I had no previous experience with React or React Native. This meant that one of the first challenges was building up enough knowledge to make sensible technical decisions and begin development.

I completed a Udemy course on React and also built on the React content covered in college. This helped me understand component-based development, state management, props, reusable UI components, and how React applications are structured.

This also influenced how the project developed. I began by learning the fundamentals of React, then gradually applied those concepts to React Native and Expo. As the project progressed, I became more comfortable separating screens into reusable components, using hooks for shared logic, and structuring the app in a way that could support future backend integration.

4.4 Parcel Wizard Service

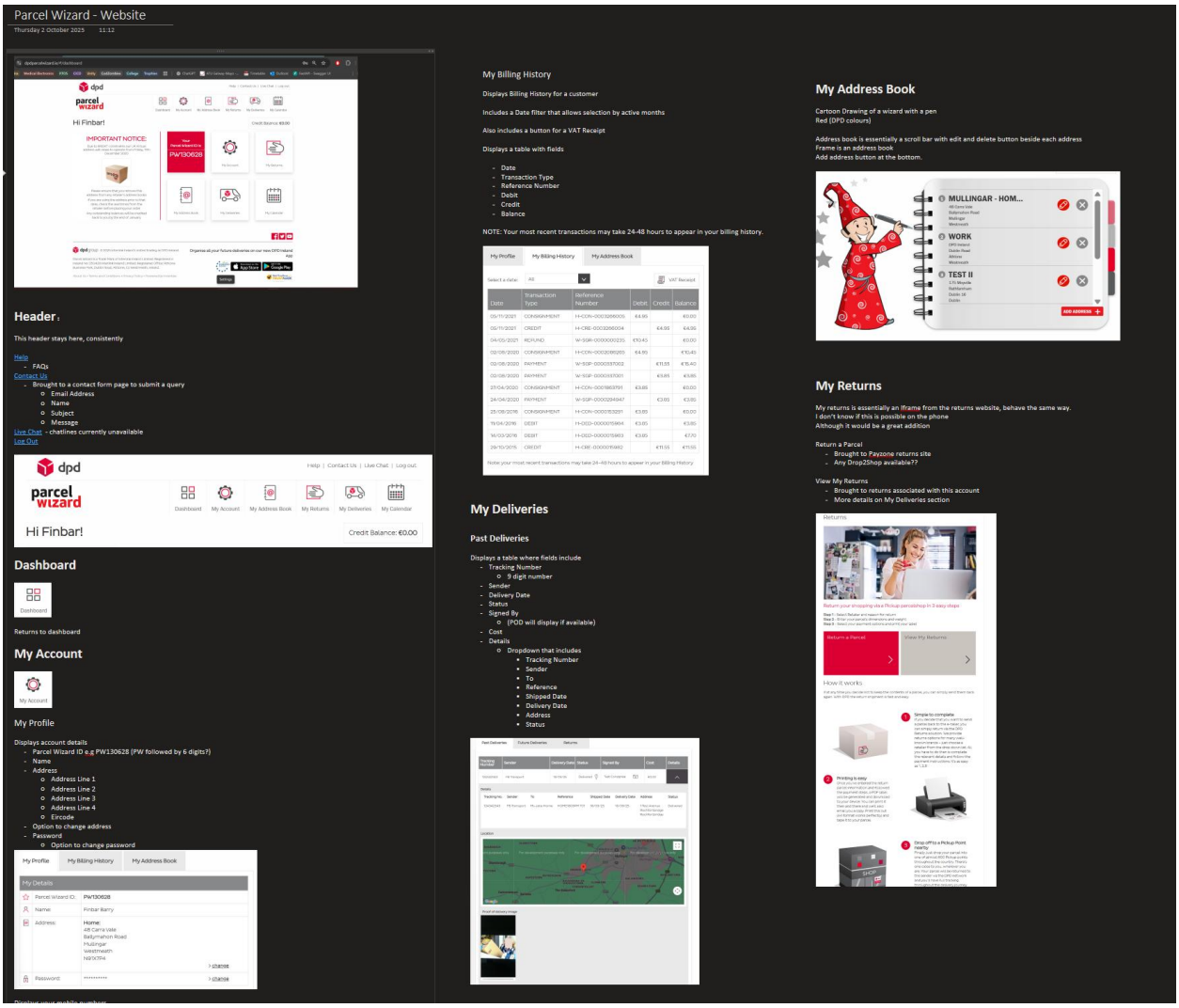
Parcel Wizard is a service that allows users to manage and customise how their DPD parcels are delivered. The service gives users control over delivery options such as choosing delivery dates, selecting delivery locations, receiving tracking updates, and viewing delivery history.

This made it a suitable project because it contains a wide range of practical app features. A Parcel Wizard app needs account management, parcel tracking, delivery management, address management, calendar scheduling, and support functionality.

4.5 Research into the Existing Parcel Wizard Website and App

During the research phase, I reviewed the existing Parcel Wizard website [1] and old app screens to understand which features were already available and which areas could be improved. The website included account management, mobile number and email management, an address book, delivery history, future deliveries, returns, and a calendar scheduling feature.

I went through and documented each of these features to understand their functionality for future reference.



I also reviewed user feedback on the old app. [2] Several reviews mentioned issues such as login problems, poor reliability, crashes, difficulty managing deliveries, and the need for the app to be modernised. This influenced the direction of the project. The aim was not only to recreate the existing Parcel Wizard functionality, but to design a cleaner and more user-friendly experience that would make common tasks easier for customers.

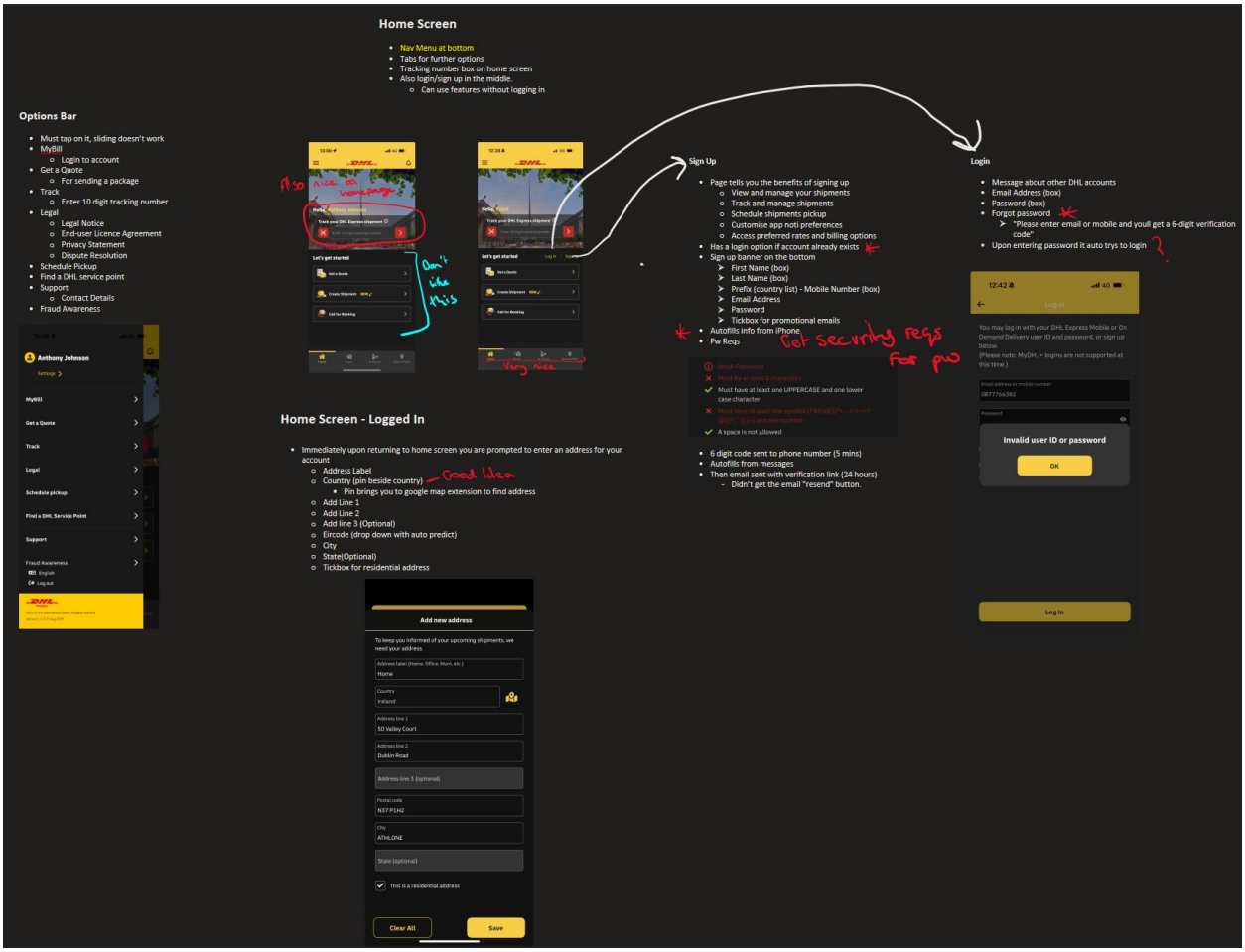
A Google user

★☆☆☆☆ 2 July 2019

Good in theory, but completely useless most of the time. Constant error messages, so you can't see the information when you need it. When the app decides it wants to work it sends multiple notifications of the same thing. I don't know if I've ever used such a frustrating app before!

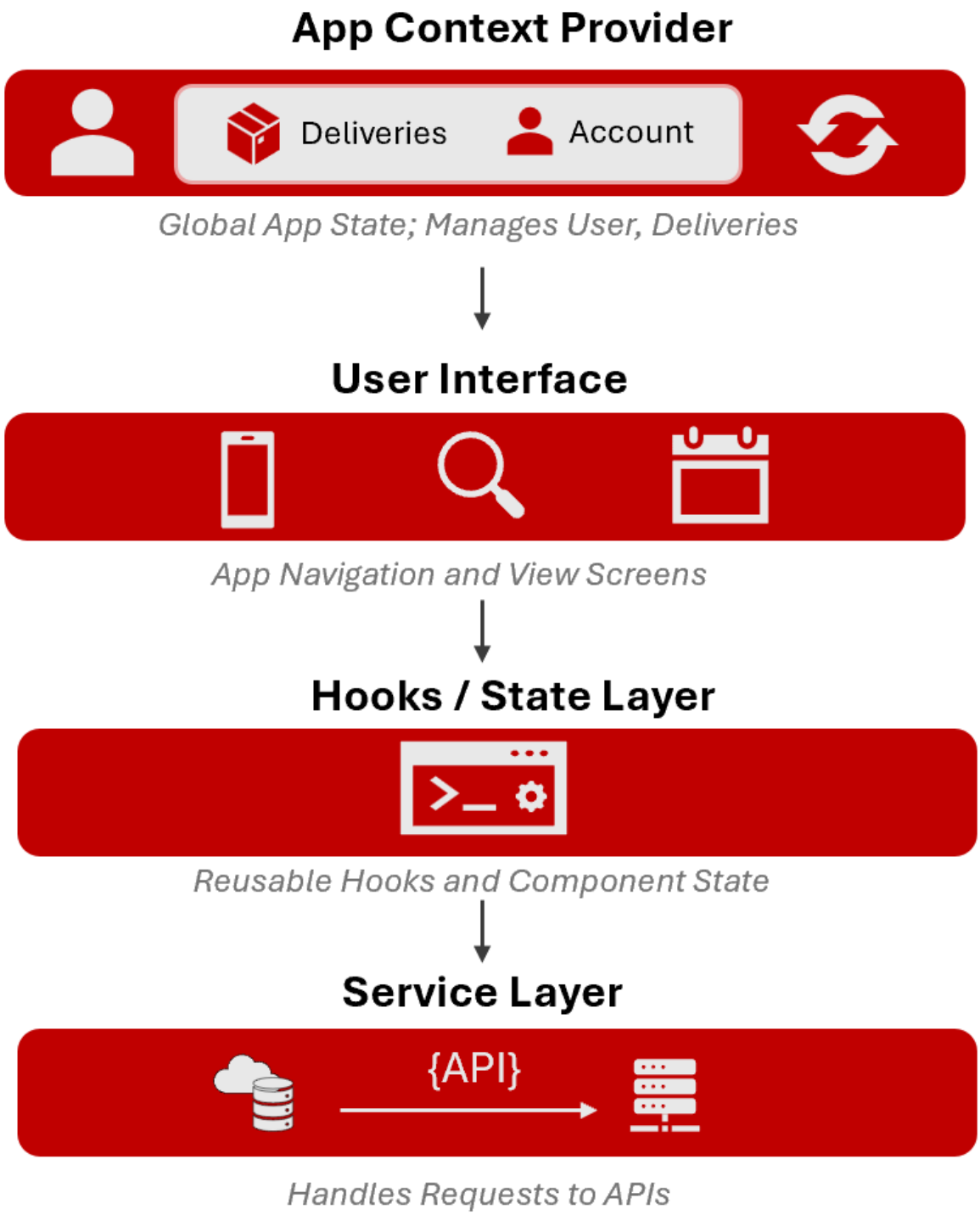
4.6 Competitor Research

Competitor research was also used to understand what users expect from modern logistics apps. I reviewed apps such as DHL and An Post to see how other delivery companies structure their mobile experiences.

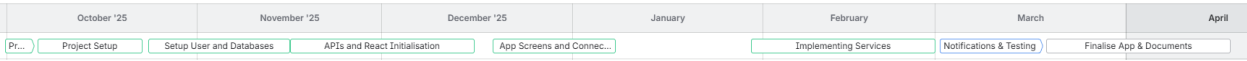


The DHL app showed the importance of having tracking available from the home screen, clear sign-up flows, address entry, service point finding, notifications, and support options. The An Post app also highlighted the value of quick access to tracking, delivery management, returns, and account-based features.

5 Project Architecture



6 Project Plan



The initial plan was to build the application against a DPD backend environment that would contain an empty database and one test account. This would have allowed the app to behave like a realistic customer-facing system without using real customer information. For GDPR and data protection reasons, I could not use DPD's test environment as it contained real-world user data.

As the project progressed, it became clear by around February that receiving and integrating with a dedicated DPD backend environment would not be achievable within the project deadline in April. I therefore repurposed a mock DPD-style backend that I had previously created for a Semester 1 project. This backend gave me control over the data structure, API responses, accounts, and consignments, while still allowing the frontend to interact with backend services in a realistic way.

This change became an important architectural decision. Instead of tightly coupling the frontend directly to the backend response format, I introduced mapping and normalisation layers. These mappers and normalisers convert backend data into the shape expected by the frontend components. This means the frontend can work with a stable internal data model, even if the backend field names or structure are different. Although the backend used in the final project is not DPD's real backend, this approach makes future integration easier because only the service, mapper, or normalisation layers should need to change.

I used Figma to design and experiment with screen layouts before building them in React Native. This helped me think through the user interface, navigation flow, and visual structure before writing code.

To manage the work, I used Jira to track project tasks throughout development. This allowed me to break the project into smaller pieces, such as authentication, account management, tracking, calendar scheduling, support, and UI polish. Using Jira helped keep the project organised and keep me organised also, as when I had long breaks between sessions, I could identify what needed to be done next.

I also kept a weekly diary in OneNote throughout the project. This was used to record progress, decisions, problems, and ideas as the project developed. Keeping the diary helped me track how the project changed over time, especially when features had to be adjusted or when technical issues were encountered. It also gave me a useful reference point when writing the final report, as I could look back at earlier planning, research notes, design decisions, and development milestones.

7 Technology Stack and Architecture

7.1 React-Native and Expo

React Native was selected as the primary development framework due to its ability to support cross-platform mobile development using a single codebase. This approach significantly reduces development time and complexity compared to building separate native applications for iOS and Android, while still providing near-native performance and access to device features. Additionally, React Native aligns well with modern JavaScript and component-based design patterns, making it suitable for building scalable and maintainable user interfaces.

Expo was used to further streamline the development process. The Expo managed workflow removes the need for complex native configuration, allowing the project to focus on functionality and user experience rather than low-level setup. Expo provides a wide range of built-in modules, including support for camera access, location services, secure storage, and haptic feedback, all of which are integrated through simple APIs.

The project makes use of Expo Go during development, enabling the application to run directly on a physical device without requiring a full native build process. This allows rapid testing and iteration, as features can be previewed instantly. Importantly, libraries such as maps and camera functionality work out of the box within Expo Go, meaning no additional native code or configuration is required. For example, the React Native Maps library automatically renders Google Maps on Android devices and Apple or Google Maps on iOS devices when used within an Expo environment, ensuring consistent cross-platform behaviour with minimal setup.

7.2 Navigation using Expo Router

7.2.1 Navigation Structure

The application uses a layered navigation structure built with Expo Router. [3] At the highest level, navigation is controlled through a root stack navigator, inside which the main authenticated area of the app is wrapped in a drawer navigator, and the drawer itself contains the bottom tab navigator. This creates a three-level structure:

STACK -> DRAWER -> TABS

This approach was chosen because it allows the app to support multiple navigation patterns at once. The stack navigator is used for top-level flows such as login and detail screens, the drawer provides access to app-wide secondary destinations such as the address book, and the bottom tabs provide fast switching between the main day-to-day features of the app.

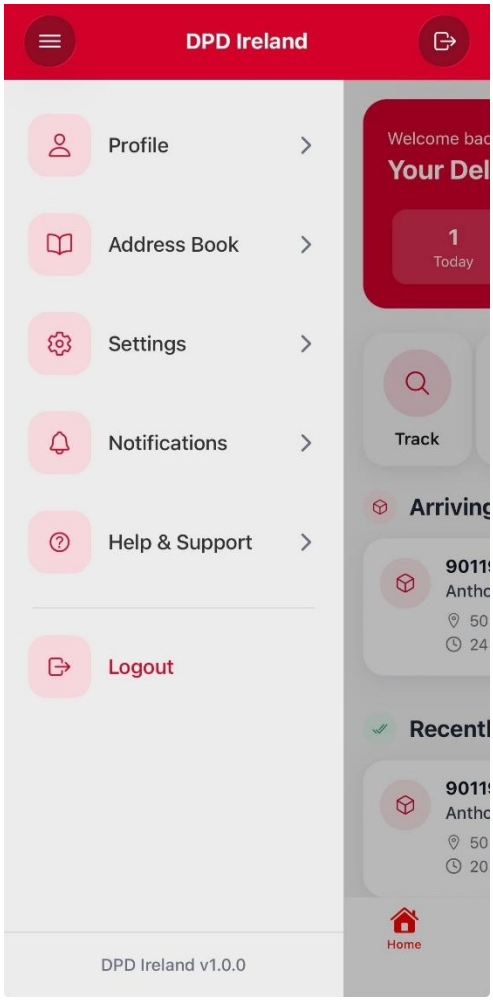
7.2.2 Root Stack Navigation

The root layout is defined in app/_layout.jsx. This file acts as the main entry point for the navigation system and wraps the app in the AuthProvider, AccountProvider, and ParcelProvider, ensuring that navigation sits inside the global state layer. The root navigator uses Stack from Expo Router and defines the main top-level routes such as login, the drawer area, scan, support, map, and parcel details.

This means that screens such as support, map, and [trackingNumber] are not part of the tab navigator itself. Instead, they are pushed on top of the main app flow as stack screens. This is because these pages behave more like full-screen detail views than permanent navigation destinations.

7.2.3 Drawer Navigation

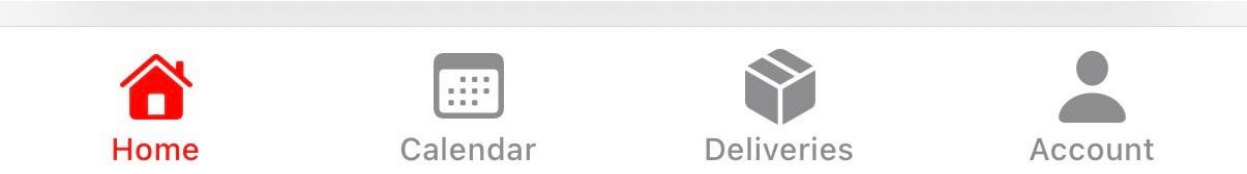
Inside the root stack, the main application area is grouped under app/(drawer)/_layout.tsx



Not all options are routed, but they represent room for expansion and new features.

The drawer [4] is configured with a custom content component.

7.2.4 Bottom Tab Navigation



The tab layout sits inside the drawer. It is defined in app/(tabs)/_layout.tsx, and the file app/(drawer)/(tabs)/_layout.tsx simply re-exports that same layout so the tab navigator [5] can be used within the drawer structure.

This design choice was made as the drawer was introduced late in the project and I had difficulty installing it. The route group inside the drawer acts as a wrapper around the existing tabs rather than creating a second independent tab system.

7.3 State Management with React Context

7.3.1 Context Architecture

The application uses React Context [6] as its main shared state management solution. Rather than passing authentication, account, and parcel data through multiple layers of components, the project centralises this data in three main providers:

- AuthProvider
- AccountProvider
- ParcelProvider

These providers are mounted at the top of the app inside app/_layout.jsx, which means all routed screens can access shared state and actions without prop drilling

```
export default function RootLayout() {
  return (
    <AuthProvider>
      <AccountProvider>
        <ParcelProvider>
          <StatusBar style="light" />
          <Stack>
            <Stack.Screen name="login" options={{ headerShown: false }} />
            <Stack.Screen
              name="(drawer)"
              options={{
```

Authentication is treated as the foundation of the app, while account and parcel data are layered on top of it. Both AccountProvider and ParcelProvider depend on values from AuthContext, so the nesting is intentional and architecturally sound.

7.3.2 AuthContext

Authentication state is managed in *authContext.jsx.* This context is responsible for:

- Managing the current auth mode
- Restoring a previous session from secure storage
- Storing the token and account number
- Exposing login, logout, and mock-mode actions

The context derives four key state values and two Booleans.

```
const [isHydrating, setIsHydrating] = useState(true);
const [mode, setMode] = useState("guest");
const [token, setToken] = useState(null);
const [accountNo, setAccountNo] = useState(null);

const isAuthenticated = mode === "live" && !!token && !!accountNo;
const isMockMode = mode === "mock";
```

7.3.2.1 Hydration

On mount, the provider tries to restore a previously saved token and account number using *getToken()* and *getAccountNo()*. If both values exist, the app restores the session and sets the mode to “live”. Otherwise, it falls back to “guest” and routes the user to the login screen.

This design allows the app to reopen into a valid logged-in state without forcing the user to log in again every time.

7.3.2.2 Login and Mode Switching

The *login()* function calls *loginToAccount()* from the auth service and then updates local context state with the returned token and account number.

```
// Login function that updates state and storage
const login = useCallback(async (accountNoInput, password) => {
  const result = await loginToAccount(accountNoInput, password);

  setToken(result.token);
  setAccountNo(result.account?.account_no || accountNoInput);
  setMode("live");

  return result;
}, []);
```

There are also two other important functions here:

- *skipLogin()* clears saved auth and switches the app into "mock" mode

```
// Skip login for mock mode
const skipLogin = useCallback(async () => {
  await clearAuth();
  setToken(null);
  setAccountNo(null);
  setMode("mock");
}, []);
```

- *logout()* clears auth and returns the app to "guest" mode

```
// Logout function that clears auth state and storage
const logout = useCallback(async () => {
  await clearAuth();
  setToken(null);
  setAccountNo(null);
  setMode("guest");
}, []);
```

The context is not only storing state, but also providing the applications overall operating mode. As this app was initially developed using mock data before switching to a backend later on in production, it was important to be able to have access to both modes whilst developing.

7.3.3 Account Context

Account related shared state is managed in *accountContext.jsx*. This is the busiest context in the app, because it is not just storing an object. It also handles:

- Loading account data
- Normalising backend responses
- Create, update and delete mutations
- Error handling
- Refresh and cache timing information

7.3.3.1 Dependency on Auth Context

The account provider depends directly on *useAuth()*

```
const { mode, isHydrating, accountNo } = useAuth();
```

This is important because account data is tied to the authenticated user. If the app is still hydrating auth state, account loading waits. If the app is in guest mode, the account is cleared. If the app is in mock mode, it loads mock account data instead.

7.3.3.2 Normalisation Layer

A design choice was made to use a *normaliseAccount()* function. The backend account shape is not used directly in the UI. Instead, the provider converts backend fields into a frontend friendly shape.

```
// Map phone_numbers to mobileNumbers and normalize field names
const mobileNumbers = Array.isArray(safe.phone_numbers)
  ? safe.phone_numbers.map((phone) => ({
      id: String(phone.id),
      value: phone.value,
      isPrimary: phone.is_primary ?? false,
    }))
  : [];
```

This separation of concerns means the UI can stay consistent even if the backend field names differ or change over time. Decisions like this were made throughout the project as the end goal for this app is to integrate with DPDs existing backend.

This kind of normalisation makes the frontend easier to maintain because all screens can work with one stable account model.

7.3.3.3 Refresh Logic

The *refreshAccount()* function handles loading current account data. Its behaviour changes depending on mode:

- In guest mode, it clears the account
- In mock mode, it loads cloned mock data
- In live mode, it fetches the account using *getAccountByNumber(accountNo)*

The provider also tracks *loading*, *isSaving, error,* and *lastLoadedAt*, which makes the state richer and more useful for the UI.

Although loading and error states are present in the code, they are not handled by a dedicated component in this project. When load times are relevant, a loading spinner could be then be displayed.

inFlightRefreshRef is used to avoid duplicate simultaneous refreshes. If a refresh is already in progress, the provider reuses the same promise instead of launching another request. This helps avoid redundant API traffic and state inconsistency.

7.3.3.4 Optimistic Mutations

The most advanced part of AccountContext is the *runOptimisticMutation()* helper. This function:

1. Stores the previous account state
2. Immediately updates the UI
3. Runs the backend API call if in live mode
4. Rolls back to the previous state if the API call fails

```
const runOptimisticMutation = useCallback(
  async (patchFn, apiFn) => {
    const previous = account;

    try {
      setIsSaving(true);
      setError(null);

      setAccount((prev) => patchFn(prev));

      if (mode === "live") {
        if (!accountNo) {
          throw new Error("Missing account number");
        }
        await apiFn();
      }
    } catch (err) {
      setAccount(previous);
      setError(err?.message || "Failed to save account change");
      throw err;
    } finally {
      setIsSaving(false);
    }
  },
  [account, mode, accountNo],
);
```

This makes the app feel fast while still preserving consistency. This helper is then used by all account mutations.

7.3.4 Parcel Context

Parcel state is managed in *parcelContext.jsx*. This provider is responsible for:

- Loading parcel data
- Switching between live and mock parcel sources
- Computing derived parcel groupings

Like AccountContext, this provider depends on AuthContext

7.3.4.1 Raw and Derived Parcel Data

The provider keeps rawParcels in state, then derives the final parcels array using *useMemo().* During that memoized transformation, it derives the expected delivery date and the delivery status, which are used for multiple UI decisions within the app.

```
const parcels = useMemo(() => {
  return rawParcels.map((parcel) => {
    const status = getParcelStatus(parcel.expectedAt);
    return {
      ...parcel,
      status,
      statusDisplay: getParcelStatusDisplay(status),
    };
  });
}, [rawParcels]);
```

7.3.4.2 Refresh Logic

The *refreshParcels()* function behaves similarly to the account refresh logic:

- If auth is still hydrating, it waits
- If mode is guest, it clears parcels
- If mock mode is active or if no account number is available, it loads *mockDeliveryDetails*
- Otherwise, it fetches real parcels using *getParcelsForAccount(accountNo)* and maps them with *mapConsignmentsToParcels()*

```
export default function mapConsignmentToParcel(consignment) {
  return {
    id: String(consignment.consignment_number),
    trackingNumber: String(consignment.consignment_number),

    toName: consignment.name,
    fromName: consignment.sender_name,

    address: {
      line1: consignment.addressline1,
      line2: consignment.addressline2 || "",
      line3: consignment.addressline3,
      line4: consignment.addressline4,
      eircode: consignment.eircode,
    },

    eta: {
      label: formatDateLabel(consignment.expected_delivery_date),
    },

    shippedAt: consignment.shipped_at,
    expectedAt: consignment.expected_delivery_date,
    status: consignment.status,
    statusDisplay: consignment.statusDisplay,

    deliveryDepot: consignment.delivery_depot,
    weight: consignment.weight,
    accountNo: consignment.account_no,
  };
}
```

This normalisation helper is essential for future integration with a different backend provider in the same way the account normaliser is.

7.3.4.3 Derived Parcel Groupings

The context exposes three important derived parcel collections:

- arrivingToday
- recentlyDelivered
- inTransit

These are computed with useMemo() by filtering the enriched parcels array by status.

That is important because it means the screens themselves do not need to repeatedly perform parcel filtering. The context becomes the single source of truth for both parcel data and the main parcel categories used by the UI.

7.3.5 Context Summary

The current context architecture is effective for several reasons.

First, it separates concerns clearly:

- AuthContext handles session state

- AccountContext handles user account data and mutations
- ParcelContext handles delivery data and parcel-specific derived state

Second, it models app behaviour cleanly across guest, mock, and live modes. This is useful in a project like this, where UI development and backend integration evolved in parallel.

And finally, it centralises refresh and mutation logic. Instead of each screen fetching and updating its own data, the providers handle data access once and expose a clean interface to the rest of the app.

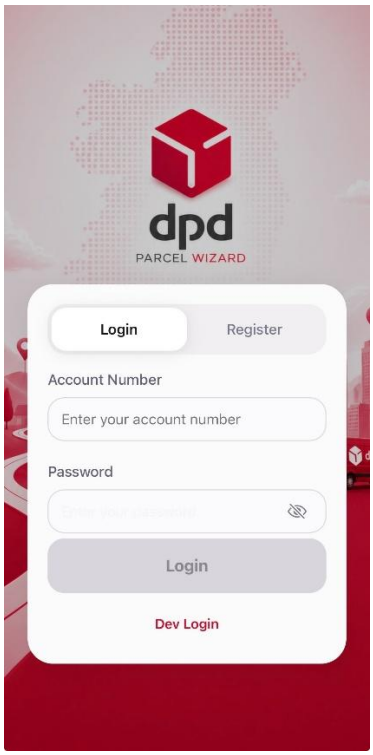
8 Authentication and Secure Storage

Since Auth Context has already been covered this section will focus on the UI flow, service calls, registration path, storage interaction, and navigation after login.

8.1 Login Flow

The login and registration system is centred around *LoginScreen.jsx,* which handles both user login and new account registration from the same screen. This mount screen uses a toggle-based interface, allowing the user to switch between Login and Register modes.

The screen background graphic is AI generated using ChatGPT. [7]



At a high level, the login flow works as follows:

1. User enters their account number and password
2. The screen calls the login function
3. The login service sends the credentials to the backend auth endpoint
4. If successful, the returned token and account number are saved securely
5. Account details are then fetched
6. The user is redirected into the main app

```
const { login, skipLogin } = useAuth();

async function handleLogin() {
  try {
    await login(loginValues.accountNo, loginValues.password);
    router.replace("/(drawer)/deliveries");
  } catch (error) {
    Alert.alert("Login failed", error.message || "Could not log in");
  }
}
```

This login handler on the screen means that the screen does not directly call the backend itself. Instead, it delegates the login to the shared login function and instead focuses on UI concerns such as error alerts and route changes.

8.1.1 Login Screen UI

The login screen is designed as a combined authentication interface. It keeps track of three groups of state:

- *loginState,* which decides whether the screen is in login or registration mode
- *loginValues & registerValues* respectively.

There is also a visibility object used to toggle password hiding for both forms.

The actual forms themselves are separated into their own components. Here we will have a look at the code in *LoginForm.jsx* as it's a bit shorter.

```
import LoginInput from "@/components/login/LoginInput";
export default function LoginForm({
  accountNo,
  setAccountNo,
  password,
  setPassword,
  hidePassword,
  onTogglePassword,
}) {
  return (
    <>
      <LoginInput
        label="Account Number"
        placeholder="Enter your account number"
        value={accountNo}
        onChangeText={setAccountNo}
        keyboardType="default"
        autoCapitalize="characters"
        autoCorrect={false}
        autoComplete="off"
      />
      <LoginInput
        label="Password"
        placeholder="Enter your password"
        value={password}
        onChangeText={setPassword}
        secureTextEntry={hidePassword}
        rightIcon={hidePassword ? "eye-off-outline" : "eye-outline"}
        onRightPress={onTogglePassword}
        autoCapitalize="none"
        autoCorrect={false}
        autoComplete="password"
      />
    </>
  );
}
```

Props are used to improve the UX by allowing the phone to recognise the input fields and store credentials for future use.

This design keeps the screen component focused on the flow logic, while the form component handles presentation.

8.1.1.1 Reusable Input Component

Both the login and registration forms use *LoginInput.jsx,* which is a reusable styled input field with support for:

- Label text
- Placeholder text
- Secure text entry
- Right-hand icon buttons

The password eye icon is implemented using a conditional right-hand Pressable, which allows the user to toggle password visibility. A small but important usability feature in authentication forms

8.1.2 Login Service Layer

The actual credential verification happens in *authService.js.* The *login()* function sends a POST request to the backend authentication endpoint:

```
export async function login(accountNo, password) {
  const response = await fetch(`${API_CONFIG.authBaseUrl}/api/auth/token`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify({
      account_no: accountNo,
      password,
    }),
  });

  const data = await response.json();

  if (!response.ok) {
    throw new Error(data.detail || "Login failed");
  }

  return data;
}
```

The backend is therefore expecting credentials in JSON format, with the account number sent under the field name account_no rather than accountNo . This shows that the frontend adapts

its request shape to match backend API requirements, just like normalisation in other areas when the roles are reversed.

If the response is unsuccessful, the function throws an error using the backend's detail field if present. This error is then surfaced back to the UI, where the login screen shows an alert dialog.

API_CONFIG variable is used here as I sometimes the ipV4 address on my laptop would change on restart. This API_CONFIG variable allows me to address the issue in one place.

8.1.3 Post-Login Processing

The more complete login flow is handled by loginToAccount() in the same file. This function does more than just request a token. After a successful login:

1. It saves the JWT access token
2. It saves the account number
3. It fetches the full account details
4. It returns both the token and the object

```
export async function loginToAccount(accountNo, password) {
  const result = await login(accountNo, password);

  await saveToken(result.access_token);
  await saveAccountNo(accountNo);

  const account = await getAccountByNumber(accountNo);

  return {
    token: result.access_token,
    account,
  };
}
```

This design decision means a successful login immediately gives the app enough information to initialise the authenticated user session properly, rather than requiring a separate follow-up step somewhere else in the UI.

8.2 Registration Flow

The login screen also supports account creation through the registration path. When the screen is toggled into register mode, it renders *RegisterForm.jsx,* which collects:

- First Name
- Last Name
- Email
- Password

Before sending anything to the backend, the screen performs one local validation check to ensure the password and confirm password fields must match. If they do not, the user immediately sees an alert.

If validation passes, *handleRegister()* constructs a full name by joining first name and last name, then calls *createAccount()* from the account service . On success, the user is shown their newly generated account number and the screen automatically switches back into login mode with that account number prefilled.

This user-friendly flow reduces friction after registration. The user does not need to remember or retype the account number manually.

8.3 Dev Login / Mock Access

As you can see from the login screen above, a "Dev Login" button is used which activates mock mode by calling skipLogin() and then routing into the app . Even though auth context was already explained, it is worth mentioning here because from a login screen perspective this acts as an alternative access path for development and testing.

This was especially useful in this project where frontend development needed to continue before backend functionality was fully complete.

8.4 Navigation After Login

Once login succeeds, the user is redirected using:

```
router.replace("/(drawer)/deliveries");
```

Here, *replace()* is used instead of *push().* That means the login screen is removed from the navigation history, so the user cannot press back and accidentally return to the login page after they have already entered the app.

8.5 Secure Storage of Credentials

The token and account number are stored in src/storage/authStorage.js using Expo Secure Store. [8] The storage layer exposes small wrapper functions for saving tokens and account numbers.

```
export async function saveToken(token) {
  await SecureStore.setItemAsync(TOKEN_KEY, token);
}

export async function getToken() {
  return await SecureStore.getItemAsync(TOKEN_KEY);
}

export async function deleteToken() {
  await SecureStore.deleteItemAsync(TOKEN_KEY);
}
```

This architecture separates persistence logic from login UI and service logic. The login screen does not need to know anything about how tokens are stored. It simply triggers login, and the service layer handles the rest.

8.6 Login Summary

The login system is built around a clear separation of responsibilities. The login screen manages user interaction, local form state, and navigation; the form components handle presentation; the auth service performs backend communication and secure persistence; and the storage layer is used for token handling. Registration is integrated into the same interface, allowing users to create an account and then move directly into the login process.

9 Account Management

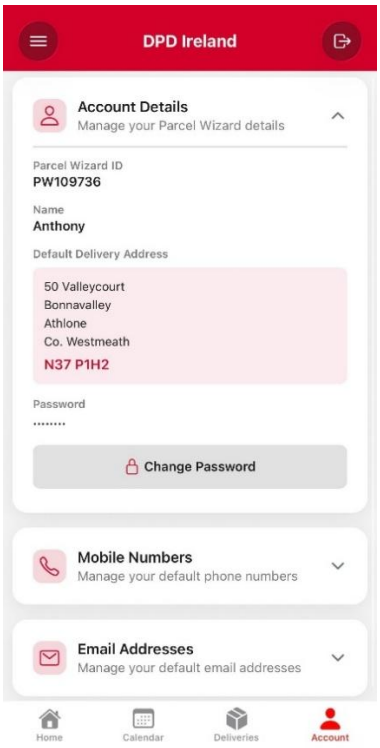
The account area is divided into two main parts. The first is the Account tab, which focuses on account details, mobile numbers, and email addresses. The second is the Address Book screen, which is accessible through the drawer and is dedicated to delivery addresses. This split was chosen because contact details and general account information are closely related, while address management is slightly larger and deserves its own space.

At the route level, the account tab is defined in *app/(tabs)/account.tsx*, while the address book drawer route renders AddressScreen through *app/(drawer)/addresses.jsx*

9.1 Account Tab Screen

The main account tab is built in *app/(tabs)/account.tsx.* It uses three collapsible dropdown sections:

- Account Details
- Mobile Numbers
- Email Addresses



This design keeps the account tab compact and easy to scan.

9.1.1 Account Dropdown Component

The collapsible wrapper itself is implemented in *AccountDropdown.jsx*.

```
return (
  <View style={styles.container}>
    <Pressable
      onPress={onToggle}
      style={({ pressed }) => [styles.header, pressed && styles.pressed]}
    >
      <View style={styles.left}>
        <View style={styles.iconContainer}>
          <Ionicons name={icon} size={24} color={Colors.dpdRed} />
        </View>

        <View style={styles.titleWrap}>
          <Text style={styles.title}>{title}</Text>
          <Text style={styles.subtitle}>{subtitle}</Text>
        </View>
      </View>

      <Ionicons
        name={isOpen ? "chevron-up" : "chevron-down"}
        size={20}
        color="#6B7280"
      />
    </Pressable>

    {isOpen && (
      <View style={styles.body}>
        <View style={styles.divider} />
        {children}
      </View>
    )}
  </View>
);
```

The dropdown header is pressable, and the open state is passed in from the parent rather than stored internally. This pattern keeps the component reusable and easy to control from the screen level.

9.1.2 Account Details Panel

Then Account Details panel implemented in *AccountDetails.jsx* reads account data through the *useAccount()* hook.

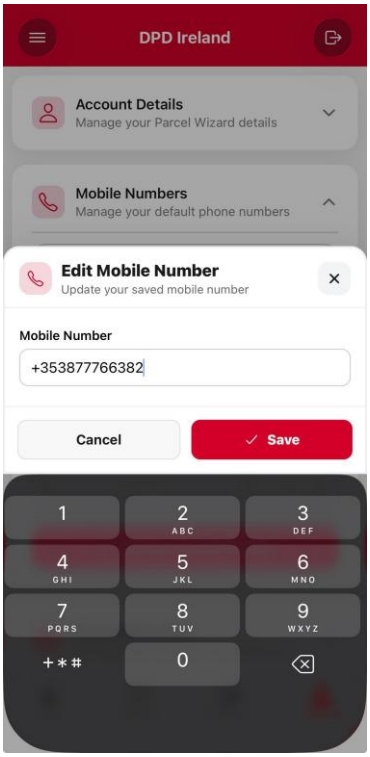
This component derives the default address by searching the account's address list for *isDefault*. If one exists it is rendered inside a styled address card.

There is also a change password button underneath the address card which is purely UI for the time being.

9.1.3 Mobile Number Management

Mobile number management is handled in *MobileNumbers.jsx*. This panel reads the list of phone numbers from *useAccount()* and renders one ContactCard per number

This component features a modal that allows users to add, edit and delete phone numbers.



9.1.3.1 New Phone Form

NewPhoneForm.jsx uses a modal bottom-sheet style layout with:

- FormHeader
- Input
- FormFooter

These components allow us to reuse this style of form throughout the app for different features.

The form automatically enforces an Irish mobile prefix of +353. It strips non-digit characters after that prefix and validates that exactly 9 digits have been entered after the prefix. If not, it shows an alert.

```
function handleChange(text) {
  if (!text.startsWith(PREFIX)) {
    text = PREFIX + text.replace("+353", "").trim();
  }

  const digits = text.slice(PREFIX.length).replace(/[^0-9]/g, "");

  setPhoneNumber(PREFIX + digits);
}
```

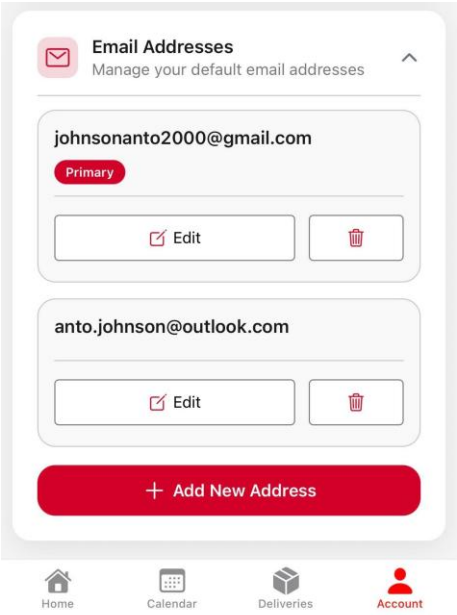
This makes the phone number form domain-specific which applies to this project as it will be deployed specifically in Ireland.

The form also auto-focuses the field when the modal opens using *useRef()*

```
const phoneRef = useRef();

function focusPhoneInput() {
  setTimeout(() => {
    phoneRef.current?.focus();
  }, 20);
}
```

9.1.4 Email Address Management

Email management follows almost the exact same structure as mobile number management. The panel is defined in *EmailAddresses.jsx*, and this time we use *NewEmailForm*.



9.1.4.1 New Email Form

NewEmailForm.jsx trims and lowercases the email before submission and validates it with a basic regular expression.

```
const emailIsValid = /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(cleanedEmail);

if (!emailIsValid) {
  Alert.alert("Invalid Email", "Please enter a valid email address.");
  return;
}
onSubmit({
  ...initialValues,
  value: cleanedEmail,
});
}
```

This is a practical level of validation for a frontend form. It does not try to fully validate all possible email formats, but it is enough to catch obvious errors before submitting.

Like the phone form, the email form opens in a modal bottom sheet and focuses the input automatically when shown.

9.2 Address Book

Address management is handled separately in AddressScreen.jsx, which is accessed through the drawer route. This screen uses *useAddresses()* and manages:

- Listing addresses
- Adding, editing and deleting addresses
- Setting a default address

The screen also stores two local UI states:

```
const [showModal, setShowModal] = useState(false);
const [editingAddress, setEditingAddress] = useState(null);
```

When the user chooses to add a new address, the modal opens with no initial values. When editing, the modal opens prefilled with the selected address.

9.2.1 Address Creation and Editing

Submission logic is handled in *handleSubmit().* If the user is editing an address, the updated form values are merged into the existing address object before calling *updateAddress().* If the user is creating a new one, the screen generates a new ID, sets a default type of "home", and marks the new address as non-default initially .

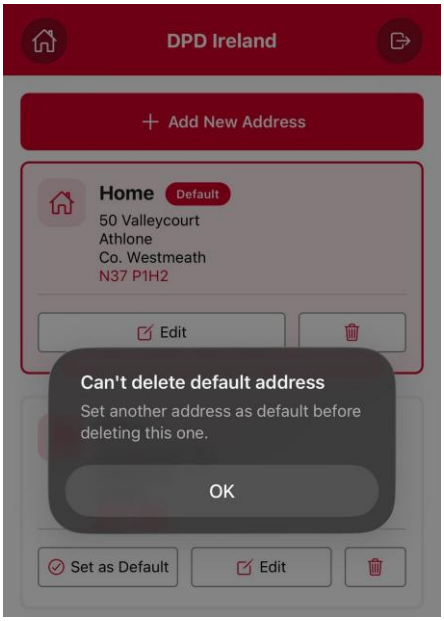
This is a practical design because it keeps the screen responsible for shaping new UI-created address objects before passing them into the hook layer.

```
function handleSubmit(formValues) {
  if (editingAddress) {
    updateAddress({
      ...editingAddress,
      ...formValues,
    });
  } else {
    const addressToAdd = {
      id: Date.now().toString(),
      type: "home",
      isDefault: false,
      ...formValues,
    };

    addAddress(addressToAdd);
  }

  handleCancel();
}
```

9.2.2 Default Address Protection

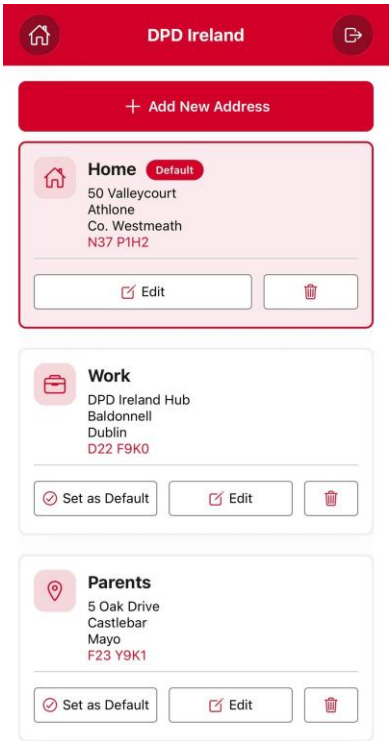
Deletion is guarded by an important rule: the current default address cannot be deleted. If the user tries, the screen shows an alert telling them to set another address as default first .

That is a strong business safeguard and prevents the account from ending up without a valid main delivery address.



9.2.3 Rendering Address Cards

Addresses are displayed using FlatList, which is more appropriate than rendering them directly in a ScrollView because it scales better for repeated list items. Each address is passed into an AddressCard component with handlers for edit, delete, and set-default actions.



The screen also handles loading and error states, returning a simple centered message when addresses are still loading or when an error occurs.

9.3 Account Management Summary

The account management feature separates responsibilities clearly.

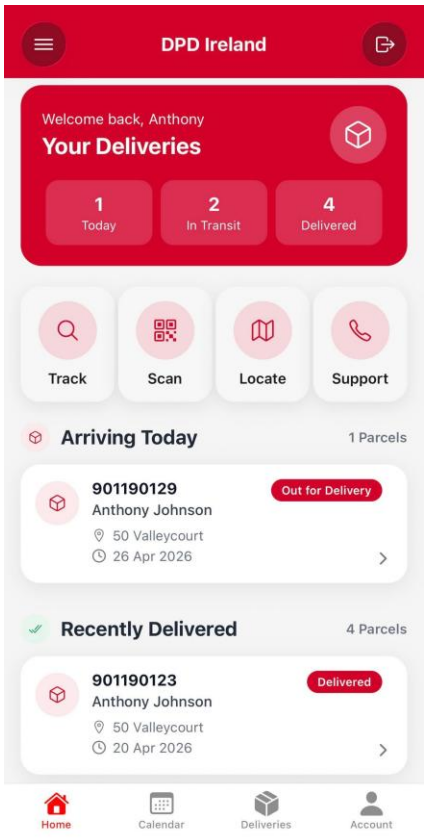
- The Account tab handles high-level account data, phone numbers, and emails
- The Address Book handles delivery addresses
- Wrapper hooks expose only the data and actions needed by each screen.
- Reusable cards and forms reduce duplication
- Modal bottoms sheets provide focused editing flows without leaving the current screen

Since these were two of the first screens completed, they lay the foundation for the styling throughout the rest of the app.

10 Home Screen and Quick Actions

The home screen is built around three main areas:

- A welcome box
- Quick Actions
- A parcel feed split into two key delivery groups



10.1 Home Screen Logic

The home screen uses the *useParcels()* hook to populate the delivery feed.

```
const { arrivingToday, recentlyDelivered } = useParcels();
```

It also manages two local state values.

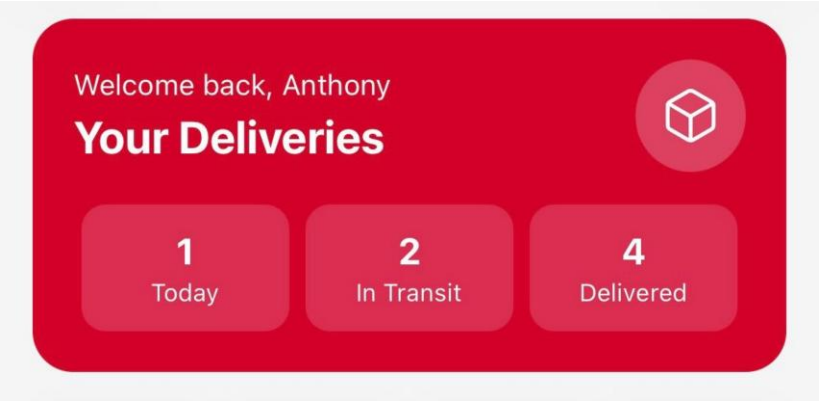
```
const [isTracking, setIsTracking] = useState(false);
const [trackingNumber, setTrackingNumber] = useState("");
```

These states control the slide-down tracking panel. The screen itself does not contain complex business logic; instead, it acts as an orchestration layer that passes parcel data and event handlers down into reusable child components.

10.2 Welcome Card

The top card is implemented in *WelcomeCard.jsx.* This component uses both the *useAccount and useParcels* hooks to fetch the users first name and three parcel statistics.

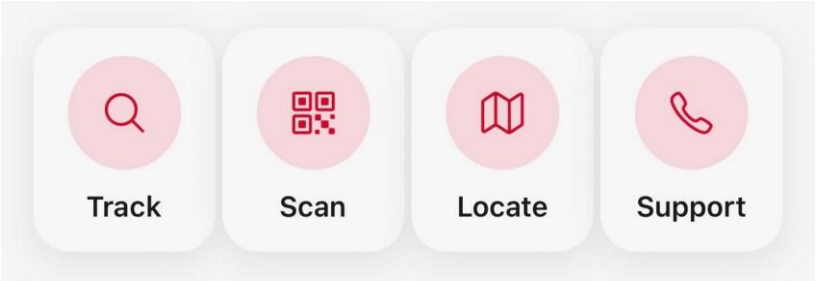
```
// Compute stats for the welcome card
const todayCount = arrivingToday.length;
const inTransitCount = inTransit.length;
const deliveredCount = recently Delivered.length;
```



This component is effective because it combines personalisation and a delivery overview in one compact element. It gives the user immediate operational information and it makes the home screen feel like a real-time reactive dashboard.

10.3 Quick Actions

Below the welcome card, the screen renders *QuickActions.jsx,* which provides four primary shortcuts. These shortcuts provide users with immediate access to the most important functions of the application without requiring them to move through multiple screens.



10.4 Track

The Track action is slightly different from the other quick actions because it does not immediately navigate to another screen. Instead, it opens a slide-down input panel on the home screen itself.

10.4.1 Home Screen Integration

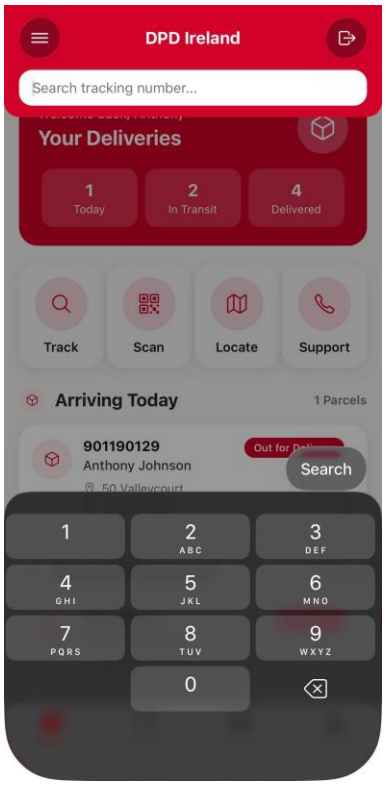
The main home screen stores:

- isTracking
- trackingNumber

When the user presses the Tracking quick action, *handleTrackingPress()* toggles the tracking panel open or closed. The panel is implemented using *SlideDownPanel,* which wraps *TrackingBox.*

```
<SlideDownPanel
  isOpen={isTracking}
  onClose={() => setIsTracking(false)}
  headerOffset={0}
  maxHeight={88}
  duration={320}
>
  <TrackingBox
    value={trackingNumber}
    onChangeText={setTrackingNumber}
    onSubmit={handleTrackSubmit}
    isOpen={isTracking}
  />
</SlideDownPanel>
```

This design is implemented in this way as tracking is treated as a lightweight, high-frequency action. The user does not need to leave the home screen to begin entering the tracking number.



10.4.2 Slide Down Panel

The slide-down panel is implemented in *SlideDownPanel.jsx*. Its purpose is to wrap another component and reveal it from the top of the screen when *isOpen* becomes true. In this case, the child component is the tracking input box, but the panel itself is reusable and could be used for other temporary UI elements in future.

The panel receives several props:

```
export default function SlideDownPanel({
  isOpen,
  onClose,
  headerOffset = 60,
  maxHeight = 400,
  duration = 380,
  children,
}) {
```

Open and close are self-explanatory, whilst *headerOffset* decides where the panel begins vertically, *maxHeight* controls how far it opens and *duration* controls how long the animation takes.

The component uses the React Native Animated API. It creates two animated values:

```
const slideAnim = useRef(new Animated.Value(0)).current;
const backdropAnim = useRef(new Animated.Value(0)).current;
```

slideAnim controls the height of the panel, while *backdropAnim* controls the opacity of the dark overlay behind it. These two animations run at the same time using *Animated.parallel().*

When the panel opens, the animated value moves from 0 to 1. That value is then interpolated into a real height:

```
const animatedHeight = slideAnim.interpolate({
  inputRange: [0, 1],
  outputRange: [0, maxHeight],
});
```

This means that when the value is 0, the panel has no height and is hidden. When the value reaches 1, the panel expands to the selected maximum height. The content is clipped using overflow: "hidden" so that the tracking box does not appear all at once before the animation has finished.

A backdrop is also rendered behind the panel. When the panel is open, the backdrop receives pointer events and can be pressed to close the panel.

10.4.2.1 Animation Difficulties

This was one of the more challenging parts of the project for me. My original plan was to use animation throughout the app to make the user experience feel smoother and more polished, and the tracking box was the first feature I wanted to animate. I didn't want the tracking input to sit permanently in the header, because I felt that would make the screen look too busy. Instead, I wanted the user to press the Track quick action and have the input slide down only when needed.

My plan was to get this first animation working properly and then apply the same approach to other parts of the app, such as forms, panels, and other temporary UI elements. However, I found the React Native Animated [9] library difficult to work with, especially when trying to animate layout properties such as height. A transform-based animation would have been simpler, but because I wanted the panel to physically open and close while clipping its contents, I had to animate height, which meant *useNativeDriver* could not be used.

The current SlideDownPanel code was created with help from AI after I struggled to get the animation working correctly myself. I kept the explanatory AI comments in the file because this was one of the few parts of the project where the implementation was not immediately obvious to me. The rest of the component structure makes sense to me now, it uses animated values, interpolates height, fades in a backdrop, and clips the child content, but getting all of those pieces working together was difficult.

I still included the feature because it was important to the experience I wanted for the home screen. The final result means that tracking stays out of the way until the user asks for it, while still being quickly accessible from the main dashboard.

10.4.3 Tracking Box

The input itself is implemented TrackingBox.jsx.

```
export default function TrackingBox({ value, onChangeText, onSubmit, isOpen }) {
  const inputRef = useRef(null);

  useEffect(() => {
    if (!isOpen) {
      inputRef.current?.blur();
      Keyboard.dismiss();
      return;
    }

    const timer = setTimeout(() => {
      inputRef.current?.focus();
    }, 200);

    return () => clearTimeout(timer);
  }, [isOpen]);

  return (
    <View style={styles.container}>
      <View style={styles.searchContainer}>
        <TextInput
          ref={inputRef}
          value={value}
          onChangeText={onChangeText}
          placeholder="Search tracking number..."
          placeholderTextColor="#777"
          keyboardType="number-pad"
          autoCorrect={false}
          autoComplete="off"
          textContentType="none"
          autoCapitalize="none"
          spellCheck={false}
          returnKeyType="search"
          maxLength={9}
          onSubmitEditing={onSubmit}
          style={styles.input}
        />
```

The use of props here enhances UX by disabling autocorrect and displaying a numeric keyboard. The component also manages focus behaviour using *useEffect().* When the panel opens, the input is focused after a short delay. When the panel closes, the input blurs and the keyboard is dismissed.

10.4.4 Submission Flow

When the user submits the tracking number, the home screen handler routes to */$[trackingNumber]* which then displays the parcel details for the entered tracking number.

10.5 Scan

The scan action routes directly to scan.jsx. This screen allows the user to scan a QR code using the device camera. In the context of the Parcel Wizard app, this feature is designed to support DPD locker where scanning a QR code can trigger an account-specific response and open the users designated collection locker.

10.5.1 Scan Screen Structure

The scan screen uses:

- CameraView and useCameraPermissions from expo-camera [10]
- Local scanned state

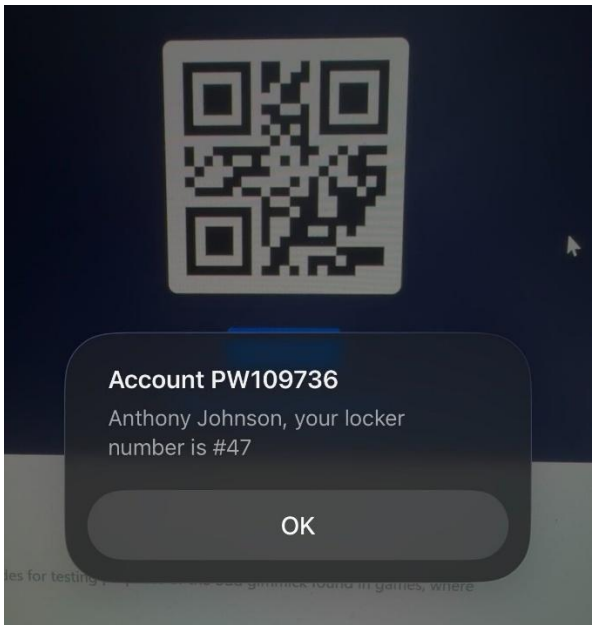
On mount, the screen checks whether camera permission exists. If not granted, it requests it. If permission is still unavailable, the screen shows a fallback message: “No camera access” .

Once permission is granted, the screen fills the page with the camera preview and enables QR scanning:

```
<CameraView
  style={StyleSheet.absoluteFillObject}
  barcodeScannerSettings={{ barcodeTypes: ["qr"] }}
  onBarcodeScanned={
    scanned
      ? undefined
      : () => {
          setScanned(true);
          Alert.alert(
            "Account " + accountNumber,
            userName + ", your locker number is #47",
            [
              {
                text: "OK",
                onPress: () => {
                  router.replace("/(tabs)/deliveries");
                },
              },
            ],
            { cancelable: false },
          );
        }
  }
/>
```

10.5.2 Scan Behaviour

The screen currently prevents repeated scans by setting scanned to true after the first detected QR code. That defensive behaviour avoids repeated triggers from the same QR code.



At the moment, the locker number is hard-coded as #47. This is acceptable for the prototype because the goal is to demonstrate the flow of a QR scan response rather than integrate with a real locker allocation system. In a production version, the scanned QR code would likely be sent to the backend, and the backend would return the correct locker number, or pickup instruction.

10.6 Locate

The Locate action is structured around a custom hook, *useLocateUser.jsx,* rather than directly embedding location logic in the button component. It uses expo-location to request the user's foreground location permission and then retrieve the device's current GPS coordinates.

This feature is designed to support the DPD location-finding part of the app. From a user perspective, the Locate quick action should be a fast way to find relevant DPD locations near them, such as depots, parcel collection points, or lockers. In the current prototype, the map shows the user's current location and a fixed DPD Head Office marker. This gives the feature a working map flow while leaving room for future expansion into multiple live DPD locations.

10.6.1 Locate Hook

Inside *locateUser(),* the flow is

1. Request foreground location permission [11]
2. Check whether permission was granted
3. Get the current coordinates
4. Navigate to */map* with the latitude and longitude as route parameters

```
const { status } = await Location.requestForegroundPermissionsAsync();

if (status !== "granted") {
  setError("Location permission was denied.");
  return;
}

const location = await Location.getCurrentPositionAsync({});

router.push({
  pathname: "/map",
  params: {
    latitude: String(location.coords.latitude),
    longitude: String(location.coords.longitude),
  },
});
```

This separation of concerns means that the quick action itself stays simple while the hook handles permission and geolocation complexity.

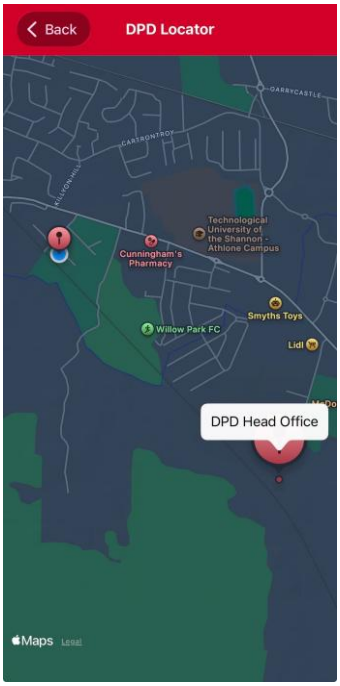
10.6.2 Map Screen

Map.jsx reads the latitude and longitude from the route parameters, converts them to numbers, validates them, and then builds a region object for Expo's React Native Maps. [12]

If the coordinates are invalid, it shows a fallback message:

```
if (!hasValidCoords || !region) {
  return (
    <View style={styles.centered}>
      <Text style={styles.message}>Location data is unavailable.</Text>
    </View>
  );
}
```

If valid it renders a full-screen MapView, with DPD locations near you pinned on the map.



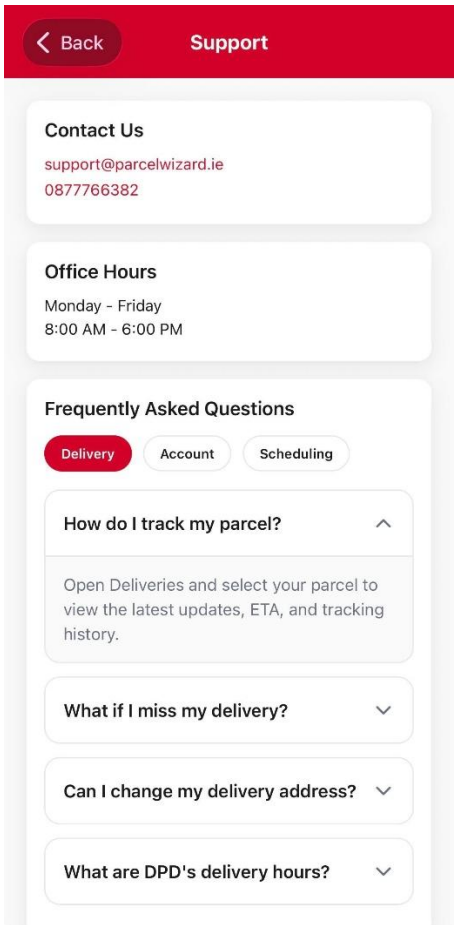
This allows the user to find locations near them where they can organise a parcel collection or an alternative delivery method.

10.7 Support

The Support action is the simplest of the four quick actions from a navigation point of view. In QuickActions.jsx, pressing Support routes directly to /support.

10.7.1 Support Screen

The support screen is implemented in support.tsx. It is built as a vertical ScrollView, with separate card sections for contact details, office hours, and frequently asked questions. This keeps the layout simple and easy to scan.



It gives the user contact details and displays the offices hours along with a tab select section on FAQs.

10.7.2 FAQs

The FAQ section gives the user a self-service area where they can look for answers before needing to contact support. The support screen stores a local state value, *selectedSection.* This

controls which FAQ category is currently active. The screen then uses *useMemo()* to calculate the list of questions from the active category.

```
const sectionItems = useMemo(
  () => getFaqItemsBySection(selectedSection),
  [selectedSection],
);
```

10.7.2.1 FAQ Categories

The FAQ data is stored separately in faqData.js. This file defines three support categories, Account, Delivery and Scheduling. Each FAQ item has an id, a section, a question, and an answer. This structure makes the FAQ system easy to extend later. For example, adding a new support topic does not require changing the support screen itself. A new item can simply be added to the FAQ data file.

10.7.2.2 FAQ Section Tabs

The FAQ categories are rendered using *FaqSectionTabs.jsx*. This component maps through the available sections and renders each one as a pressable pill-shaped tab. The currently selected tab is styled differently using the DPD red brand colour, while inactive tabs use a lighter background and border.

The tabs are placed inside a horizontal ScrollView, which means more categories can be added later without breaking the layout on smaller phone screens.

```
return (
  <View style={styles.wrapper}>
    <ScrollView
      horizontal
      showsHorizontalScrollIndicator={false}
      contentContainerStyle={styles.content}
    >
      {sections.map((section) => {
        const isSelected = section.id === selectedSection;

        return (
          <Pressable
            key={section.id}
            onPress={() => onSelectSection?.(section.id)}
            style={({ pressed }) => [
              styles.pill,
              isSelected ? styles.pillSelected : styles.pillIdle,
              pressed && styles.pressed,
            ]}
          >
            <Text
              style={[
                styles.pillText,
                isSelected ? styles.pillTextSelected : styles.pillTextIdle,
              ]}
            >
              {section.label}
            </Text>
          </Pressable>
        );
      })}
    </ScrollView>
  </View>
);
```

10.7.2.3 FAQ Accordion List & Item

The actual list of questions is rendered using FaqAccordionList.jsx. This component receives the filtered FAQ items for the selected category and renders each item using FaqAccordionItem.

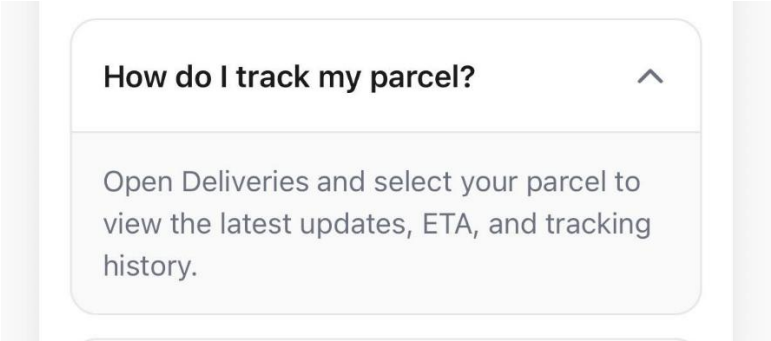
The accordion list stores an array called *openIds.* This array tracks which FAQ questions are currently open. When the user taps a question, the component either adds that question's id to *openIds* or removes it if it is already open.

This means multiple FAQ answers can be open at once. That is useful because a user may want to compare answers or keep one answer visible while opening another.

A small but important detail is the *resetKey* prop. When the selected FAQ category changes, *FaqAccordionList* resets openIds back to an empty array. This prevents answers from one category staying open when the user switches to another category.

Each individual question is rendered by *FaqAccordionItem.jsx.* The component displays a pressable question row with the question text on the left and a chevron icon on the right.

When the item is open, the answer is displayed underneath the question in a separate answer area. The answer area uses a top border and slightly different background colour, making it visually separate from the question row.

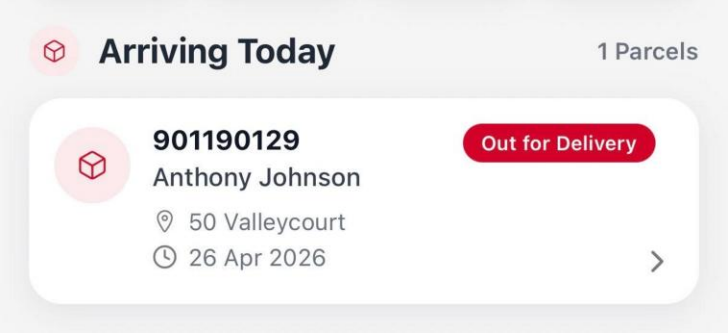


10.8 Delivery Feed

The bottom of the home screen is handled by *DeliveryFeed.jsx.* This is populated using the *useParcels* hook and renders two subsections.

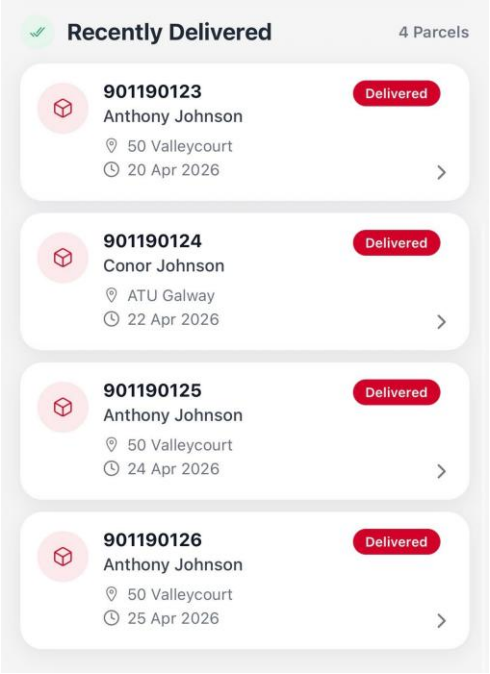
10.8.1 Arriving Today Section

ArrivingToday.jsx renders a section header and a list of *ParcelCard* components. The section header displays a dynamic parcel count. Then each parcel is rendered as a pressable card that opens parcel details using the tracking number route.



10.8.2 Recently Delivered Section

RecentlyDelivered.jsx follows the same structure as Arriving Today.



10.8.3 Parcel Card

Both sections use *ParcelCard.jsx.* This works well on the home screen because it gives enough detail to identify the parcel quickly without overwhelming the user with full parcel information.

10.9 Home Screen Summary

The home screen was designed to balance summary, action and detail. This creates a screen that feels practical from the moment it opens. A personalised summary section gives the user an immediate understanding of their current delivery activity, while a row of quick actions supports fast access to the most frequently used features. This is then complemented by a structured parcel feed that provides more detailed delivery information without overwhelming the user. As a result, the screen functions as an effective dashboard, offering both a high-level overview and direct access to day-to-day delivery tasks.

11 Parcel Details

11.1 Fetching Consignment Data

The parcel details view is implemented as a dynamic route in app/[trackingNumber].jsx. This means the screen is opened using a parcel's tracking number in the URL, allowing the same page component to display different parcels depending on the selected route parameter

The screen is built around a straightforward flow:

```
const { trackingNumber } = useLocalSearchParams();
const { parcels, loading } = useParcels();

const [isSupportOpen, setIsSupportOpen] = useState(false);

const delivery = useMemo(() => {
  return parcels.find(
    (parcel) => parcel.trackingNumber === String(trackingNumber),
  );
}, [parcels, trackingNumber]);
```

It will read the trackingNumber from the route and then find the matching parcel from the parcels object passed on by the *useParcels* hook.

11.1.1 Loading and Empty States

Before rendering the main content, the screen handles two basic states:

- If parcels are still loading, it returns an empty ScrollView
- If no matching parcel is found, it renders a basic empty state with the message "Parcel not found"

That keeps the screen safe from crashing when data is unavailable, although the loading state is minimal and could be made more user-friendly in future implementations with a proper loading indicator.

11.2 Parcel Detail View and Tracking History

11.2.1 Main Screen Composition

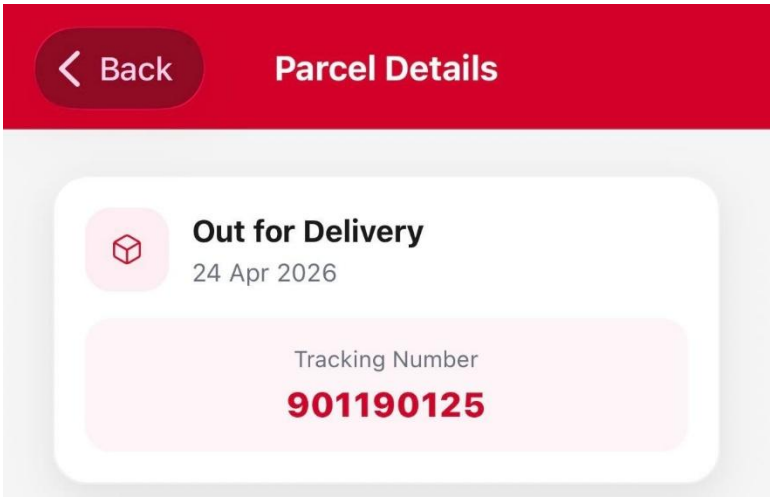
Once the parcel is found, the screen builds the page from several reusable components.

- StatusSummary
- SectionCard
- InfoRow
- PodImage
- TrackingHistory

This is a strong component-based design because each part of the parcel details page has a single responsibility. The screen itself mostly coordinates data and conditional rendering, while the smaller components handle layout and styling.

11.2.2 Status Summary

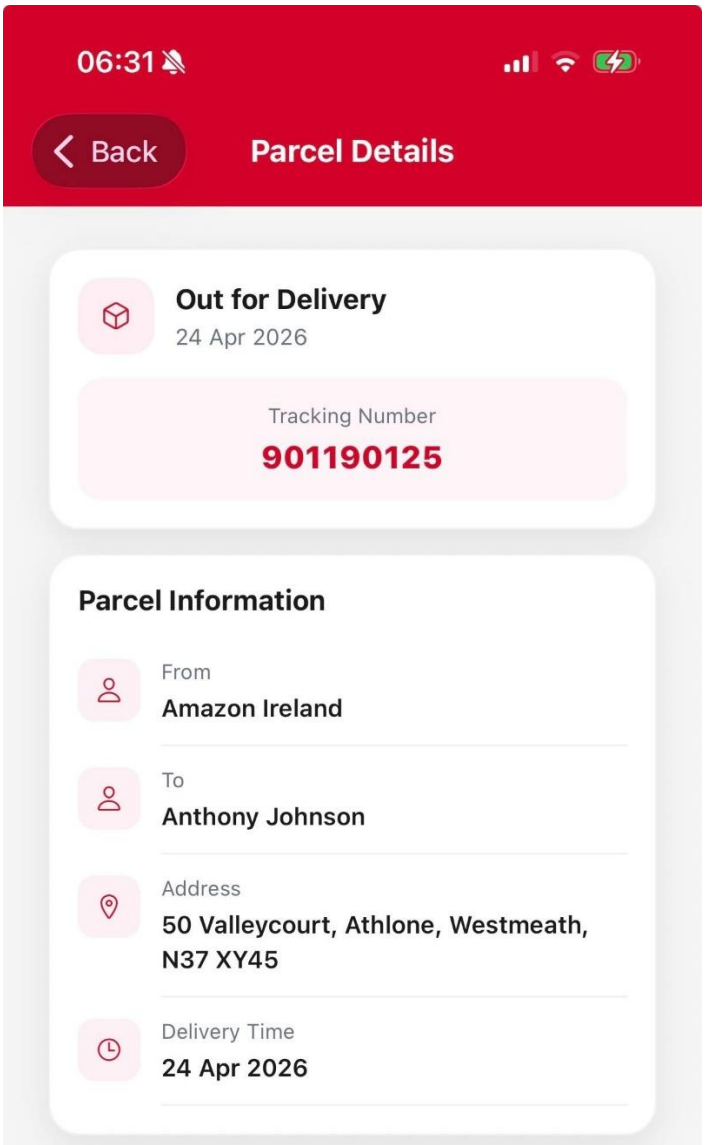
StatusSummary.jsx displays the parcels current status, along with the ETA label and the tracking number.



This works well because the most important information is placed at the top of the page and visually separated from the rest of the parcel details.

11.2.3 Parcel Information Section

The “Parcel Information” section is rendered using SectionCard plus several InfoRow components.

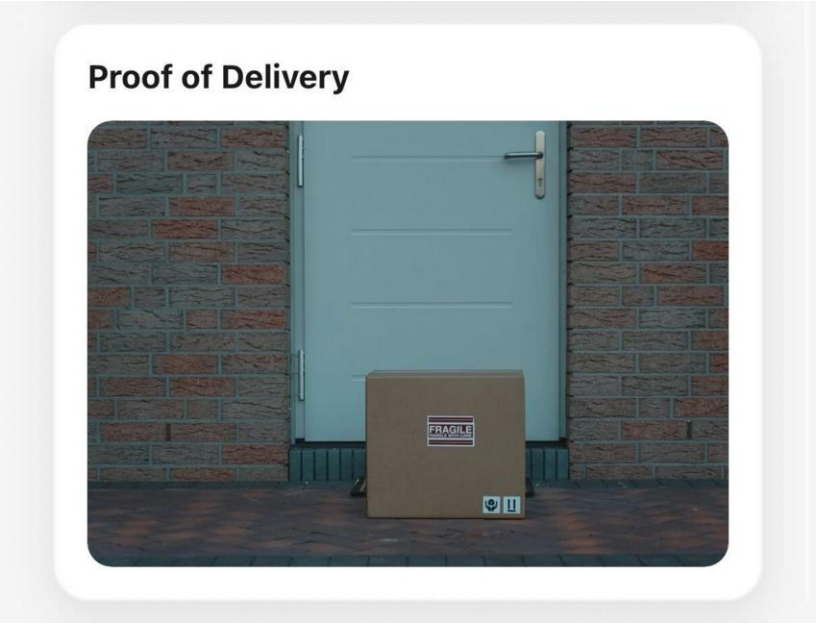


This is a clean pattern because it makes the parcel information section easy to extend if more parcel metadata is needed later.

11.2.4 Proof of Delivery

Because this is not a live application, I made the decision to conditionally render a default POD image if the parcel status is delivered. A POD image is a core part of the logistics experience, so

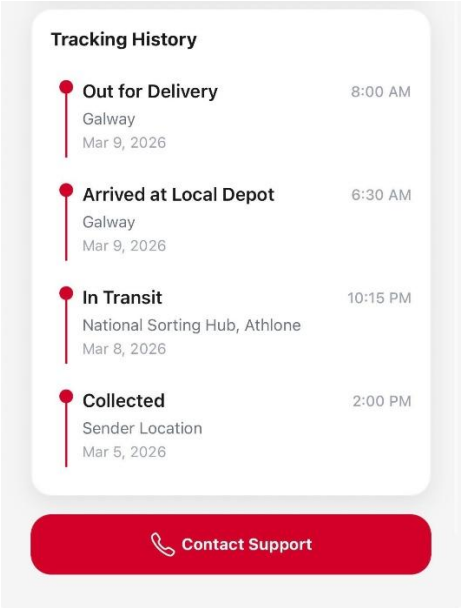
showing a placeholder ensures the UI remains consistent, communicates completion to the user, and avoids leaving an empty or broken visual state. It also lets us validate the end-to-end flow without depending on real POD assets or backend integrations.



11.2.5 Tracking History

Like the POD image, tracking history is rendered based on the status of the parcel. The current screen selects one of two mock tracking-event arrays.

- deliveredParcelTracking
- outForDeliveryTracking



This is a reasonable approach for the project because it allows the full timeline UI to be built and styled without the need for event level backend integration.

TrackingHistory.jsx itself is essentially just a thin wrapper around SectionCard and TimelineList.

11.2.5.1 Timeline List

TimelineList.jsx maps each event to a TimelineItem component.

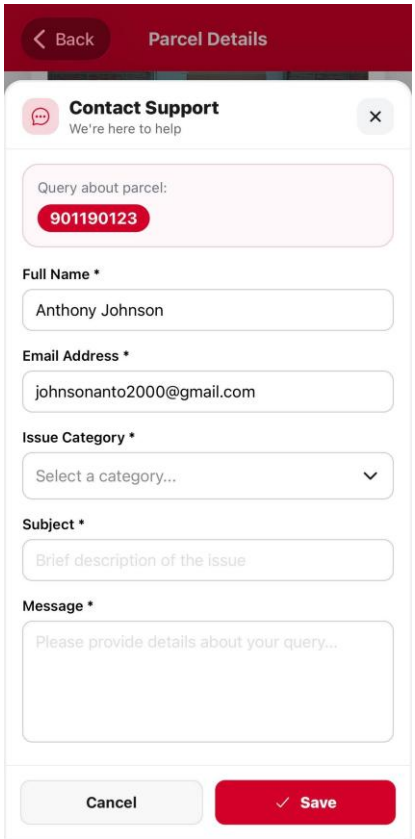
```
<View>
  {events.map((event) => (
    <TimelineItem
      key={event.id}
      status={event.status}
      location={event.location}
      date={event.date}
      time={event.time}
    />
  ))}
</View>
```

This makes event rendering fully reusable and scalable.

11.2.6 Parcel Support

At the bottom of the screen there is a red “Contact Support” button. Pressing it opens a parcel-specific support form by toggling the local isSupportOpen state.

This ParcelSupportForm.jsx is a modal bottom-sheet style form that is prefilled with the active parcel and account details.



The form then asks the user to fill in the issue category, a subject and message. If validation succeeds, it builds a payload object and passes it back to the parent *onSubmit* handler. Currently, as I don’t have a support team, I don’t do anything with this payload. But this kind of experience that allows a user to report an issue easily is invaluable to both themselves and DPD.

11.3 Parcel Details Summary

The parcel details screen was designed as a structured, component-based view that presents the most important delivery information in a clear hierarchy. A dynamic route is used to load

the selected parcel, while reusable cards and timeline components organise status information, parcel metadata, proof of delivery, and tracking history into distinct sections. The screen also supports a parcel-specific support workflow through a prefilled modal form, extending the page beyond passive viewing into practical customer assistance.

12 Calendar and Scheduling

The calendar feature is implemented as a dedicated screen that combines parcel visibility with delivery scheduling functionality. Its purpose is twofold: first, to give the user a clear month-based overview of upcoming parcel deliveries, and second, to allow the user to create delivery schedules for selected dates. The overall structure of the module is cleanly separated into:

- The main screen (*CalendarScreen.jsx)*
- A screen logic hook *(useCalendarScreen.jsx)*
- A schedule hook *(useSchedules.jsx)*
- Presentational calendar components such as CalendarCard, MonthHeader, WeekdayRow, DaysGrid, and DaysCell
- Schedule UI components such as SelectDatesCard and ScheduleDetailsCard
- Shared date utilities from *utils/date*

This separation makes the feature easier to maintain, because rendering, interaction logic, and scheduling data are not all mixed into one file.

12.1 Main Calendar Screen

The feature begins in CalendarScreen.jsx. This screen is responsible for coordinating the entire calendar flow. It imports the screen state from *useCalendarScreen(),* schedule state from *useSchedules(),* then passes those values into the visual components that render the calendar and schedule forms.

At the top of the screen the useCalendarScreen hook provides the state needed to populate the calendar.

```
const {
  selectedDate,
  monthLabel,
  days,
  deliveryDates,
  onPrevMonth,
  onNextMonth,
  onSelectDate: onSelectDateHook,
  selectedDateParcels,
} = useCalendarScreen();
```

The screen also manages local UI state for schedule creation:

```
const [isCreatingSchedule, setIsCreatingSchedule] = useState(false);
const [selectedScheduleDates, setSelectedScheduleDates] = useState([]);
const [showScheduleDetails, setShowScheduleDetails] = useState(false);
```

This allows the screen to have two operating modes.

1. Normal calendar browsing mode
2. Schedule creation mode

This design choice allows the same calendar interface to support both viewing and scheduling without needing a separate screen.

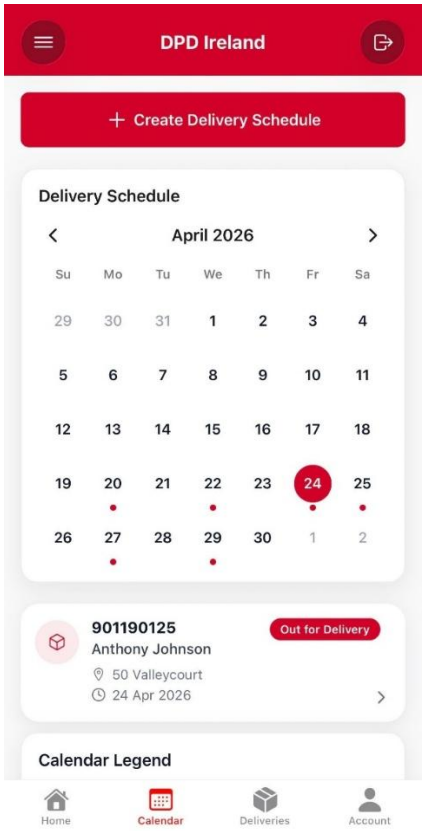
12.1.1 Calendar Interaction Flow

In normal mode, tapping a date updates the selected calendar date and filters parcels for that day. In schedule creation mode, tapping a date adds or removes it from a list of selected schedule dates instead. This behaviour is handled inside *handleSelectDate()* using *isSameDay()* from the date utilities.

This implementation reuses the same calendar grid for two different types of interaction instead of duplicating UI.

12.1.2 Rendering selected-day parcels

Below the calendar itself, the screen renders a ParcelCard for each parcel due on the currently selected date. Pressing a parcel card navigates to the parcel details page using the tracking number route.



This creates a useful connection between the calendar and delivery-tracking parts of the app.

12.2 Calendar Screen Hook

Originally, the calendar logic was kept on CalendarScreen.jsx. Because this is a busy page with lots going on, the screen became cluttered, so I moved the logic in a custom hook. Now, most of the calendar's core logic is kept in src/hooks/useCalendarScreen.jsx. This hook is responsible for:

- Storing the visible month
- Storing the selected date
- Generating the calendar day grid

- Computing which dates contain deliveries
- Filtering parcels for the selected day

12.2.1 Visible Month and Selected Date

The hook initialises today once with *useMemo(),* then creates:

- visibleMonth, starting at the first day of the current month
- selectedDate, starting as today

```
const today = useMemo(() => new Date(), []);
const [visibleMonth, setVisibleMonth] = useState(startOfMonth(today));
const [selectedDate, setSelectedDate] = useState(today);
```

This is a sensible structure because the visible month and selected date are related but not identical. A user might be viewing a different month while still having a particular day selected.

12.2.2 Delivery Date Set

The hook obtains parcels from *useParcels()* and builds a Set of expected delivery dates. This is then passed down to the calendar cells so that delivery days can be visually marked with a dot.

```
const deliveryDates = useMemo(() => {
  return new Set(
    parcels
      .filter((parcel) => parcel.expectedAt)
      .map((parcel) => parcel.expectedAt),
  );
}, [parcels]);
```

The use of Set here is a performance choice so that the calendar can perform fast look ups for whether a date contains a delivery.

12.2.3 Selected Date Parcels

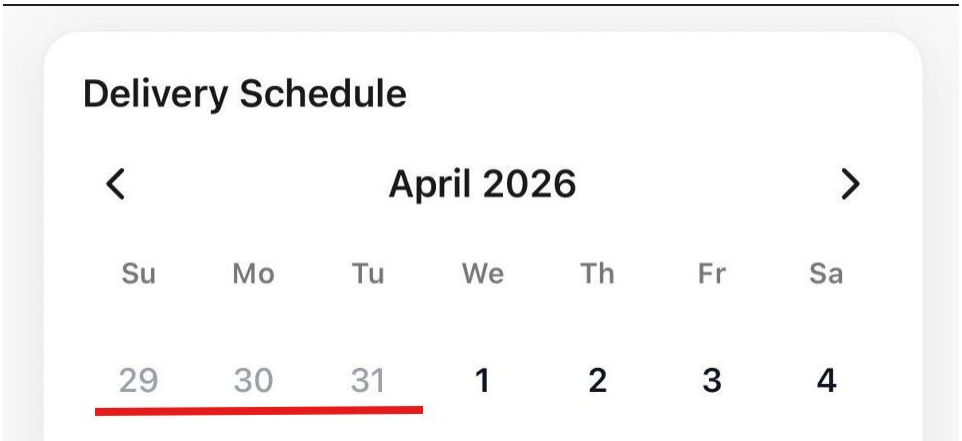
The hook also computes selectedDateParcels by converting the current selected date into a key with formatDateKey() and then filtering the parcel list to match it .

This keeps date comparison logic out of the screen component and ensures the selected-day parcel list is always derived consistently from the same key format used elsewhere.

12.2.4 Month Navigation

Previous and next month actions are implemented as callbacks. Moving backward manually constructs the previous month, while moving forward uses the shared addMonths() date utility.

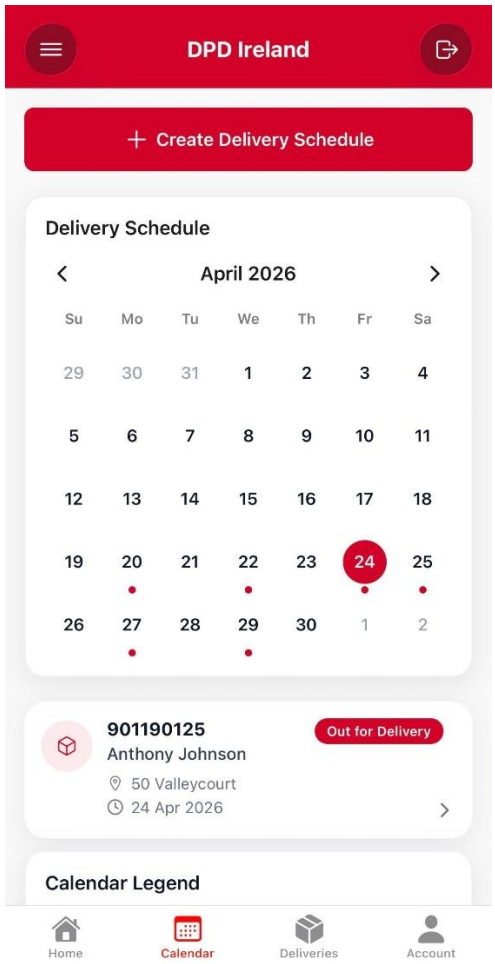
The hook also updates visibleMonth automatically if the user selects a date that belongs to the adjacent month, which is useful when tapping a trailing or leading day shown in the calendar grid.



12.3 Calendar Container Component

The main visual wrapper for the calendar is CalendarCard.jsx. This component does not contain business logic. Instead, it receives all the required props and passes them down to the smaller calendar elements:

- MonthHeader
- WeekdayRow
- DaysGrid



CalendarCard acts as the styled shell of the calendar while leaving behaviour to the screen and hooks.

12.3.1 Month Header and Navigation

The calendar header is broken into two components:

- MonthHeader.jsx
- MonthNav.jsx

MonthHeader renders the month label with left and right navigation buttons and delegates the actual button UI to MonthNav.

```
export default function MonthHeader({ monthLabel, onPrev, onNext }) {
  return (
    <View style={styles.monthRow}>
      <MonthNav name="chevron-back-outline" onPress={onPrev} />
      <Text style={styles.monthText}>{monthLabel}</Text>
      <MonthNav name="chevron-forward-outline" onPress={onNext} />
    </View>
  );
}
```

This keeps the component tree small and reusable. MonthNav itself is a simple pressable icon button using Ionicons [13].

12.3.2 Weekday Row

WeekdayRow.jsx renders the weekday headings using a fixed array:

```
const DAYS_OF_WEEK = ["Su", "Mo", "Tu", "We", "Th", "Fr", "Sa"];
```

These labels are displayed in a simple horizontal row above the day grid. The implementation is intentionally lightweight, which is appropriate because this part of the UI is static.

12.3.3 Day Grid and Day Cells

The day grid is handled in two layers:

- DaysGrid.jsx
- DaysCell.jsx

12.3.3.1 DaysGrid

DaysGrid receives the generated days array and maps it to one DaysCell per calendar entry. It does not calculate anything itself, which makes it a pure presentational grid container.

```
export default function DaysGrid({
  days = [],
  selectedDate,
  onSelectDate,
  deliveryDates,
  scheduleMap,
  selectedDates,
  isSelectingSchedule = false,
}) {
  return (
    <View style={styles.daysGrid}>
      {days.map((day) => (
        <DaysCell
          key={`${day.date.toISOString()}-${day.inMonth}`}
          item={day}
          selectedDate={selectedDate}
          onPress={onSelectDate}
          deliveryDates={deliveryDates}
          scheduleMap={scheduleMap}
          selectedDates={selectedDates}
          isSelectingSchedule={isSelectingSchedule}
        />
      ))}
    </View>
  );
}
```

12.3.3.2 DaysCell

DaysCell is where most of the day-specific visual logic happens. Each cell determines:

- Whether it matches the current selected date
- Whether it belongs to the current month
- Whether it has a delivery
- Whether it has a schedule action symbol
- Whether it is selected as part of schedule creation

The component builds a date key using *formatDateKey(item.date),* then checks:

```
const hasDelivery = item.inMonth && deliveryDates?.has(dateKey);
const scheduleSymbol = scheduleMap?.get(dateKey);
```

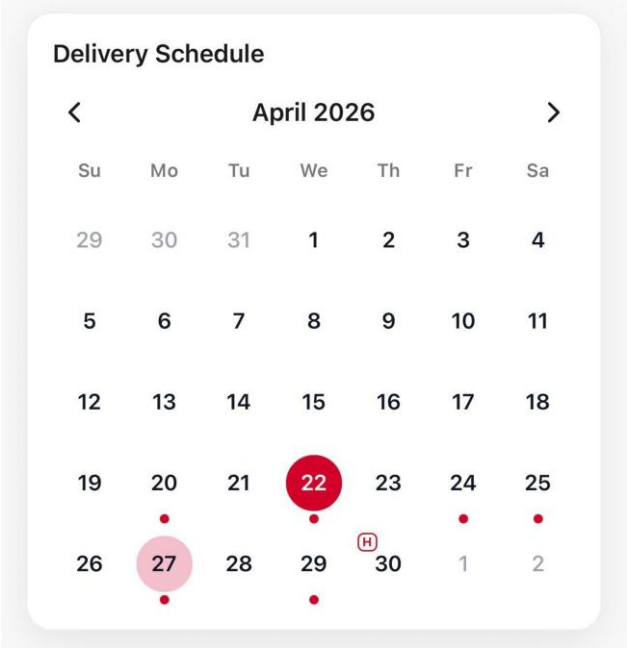
All day decorations are driven by simple lookups rather than repeated complex lookups.

12.3.3.3 Visual States in the Day Cell

The cell supports multiple visual states:

- Muted text for days outside the visible month
- Highlighted styling for today

- Red-filled styling for the selected date
- Lighter red styling for dates selected during schedule creation
- A red dot for parcel deliveries
- A small schedule badge for delivery actions such as hold, redirect, or collect



This layered visual design is one of the strongest parts of the calendar implementation, because it lets the user distinguish multiple types of information on the same day cell at a glance.

12.4 Schedule Creation

The schedule creation UI allows the user to create delivery instructions directly from the calendar screen. This feature turns the calendar from a passive delivery overview into an interactive delivery management tool. The user can select future dates and then choose what DPD should do with parcels on those dates.

The schedule creation flow is managed in CalendarScreen.jsx. The screen uses a small set of local state values to control the current stage of the schedule process:

```
const [isCreatingSchedule, setIsCreatingSchedule] = useState(false);
const [scheduleMode, setScheduleMode] = useState("single");
const [selectedScheduleDates, setSelectedScheduleDates] = useState([]);
const [showScheduleDetails, setShowScheduleDetails] = useState(false);
```

These states allow the calendar to switch between normal calendar browsing and schedule creation mode. In normal mode, tapping a date selects it and filters parcels for that day. In schedule creation mode, tapping a date adds or removes it from the list of selected schedule dates.

12.4.1 Create Delivery Schedule Button

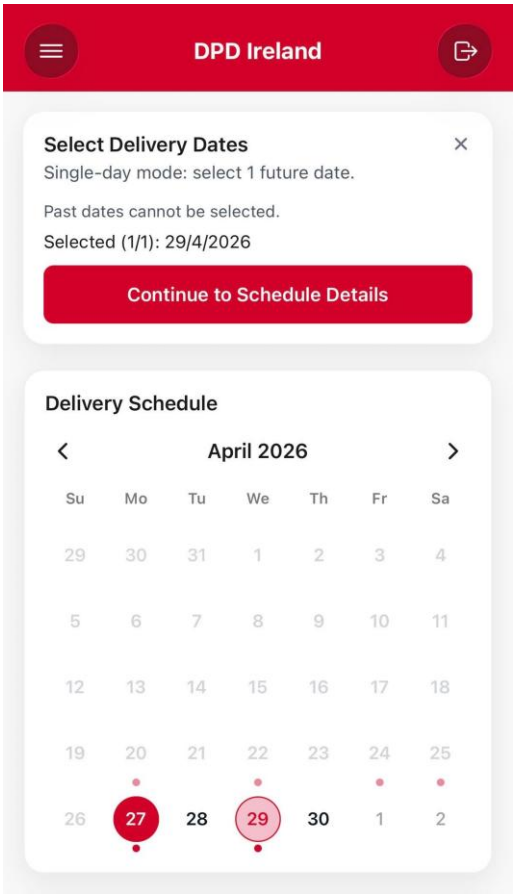
When the user is not creating a schedule, the screen displays a red Create Delivery Schedule button above the calendar. Pressing this button calls *toggleCreateSchedule(),* which switches schedule creation mode on and resets the schedule state.

```
function toggleCreateSchedule() {
  setIsCreatingSchedule((prev) => !prev);
  setScheduleMode("single");
  setSelectedScheduleDates([]);
  setShowScheduleDetails(false);
}
```

The default schedule mode is "single", meaning the user begins by selecting one future date.

12.4.2 Date Selection Mode

When schedule creation starts, the screen renders SelectDatesCard.jsx. This card sits above the calendar and explains what the user should do next.



The card prompts the user to select a start date from the calendar below and then displays that date. Past day cells are also disabled and cannot be chosen.

12.4.3 Continue to Schedule Details

Once the user has selected a date, pressing “Continue to Schedule Details” moves the flow to the second stage.

At this point, CalendarScreen.jsx stops rendering SelectDatesCard and instead renders ScheduleDetailsCard. This creates a simple two-stage flow:

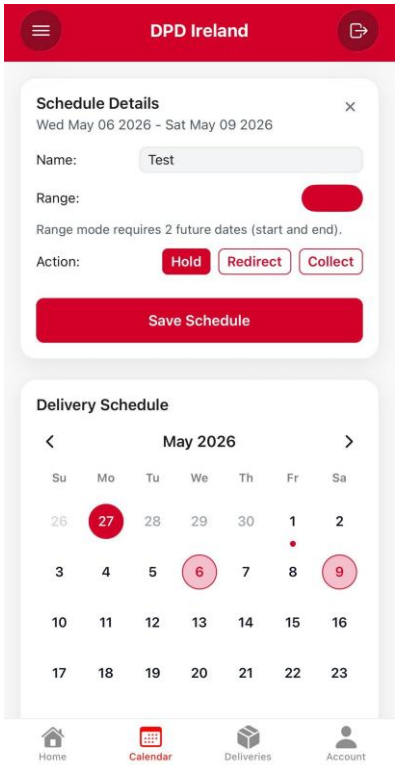
1. Select the date or date range
2. Fill in the schedule details and action

This is easier for the user than showing every option at once. It reduces clutter on the calendar screen and keeps the user focused on one task at a time.

12.4.4 Schedule Details Card

ScheduleDetailsCard.jsx is the second part of the schedule creation flow. It receives the selected dates, the current schedule mode, and callback functions for saving, cancelling, and changing mode.

The card displays the selected date or date range at the top.



The name field lets the user label the schedule. If the user leaves it empty, the app automatically creates a default name using the selected date.

12.4.4.1 Single-Day and Range Mode

The details card contains a switch that controls whether the schedule is for a single day or a range. This is connected back to the parent screen through onModeChange

If the switch is enabled, the schedule mode becomes "range". If disabled, it becomes "single".

When the mode changes back to single-day mode, CalendarScreen.jsx trims the selected date list down to one date. It sorts the selected dates and keeps the earliest one:

```
function handleModeChange(nextMode) {
  setScheduleMode(nextMode);

  if (nextMode === "single") {
    setSelectedScheduleDates((prev) => {
      if (prev.length <= 1) {
        return prev;
      }

      const ordered = [...prev].sort((a, b) => a - b);
      return [ordered[0]];
    });
  }
}
```

This prevents the state from becoming inconsistent. For example, the user cannot have two selected dates while the schedule is in single-day mode.

12.4.5 Delivery Actions

The schedule form allows the user to choose one of three delivery actions:

- Hold
- Redirect
- Collect

Internally these actions are stored as short action codes:

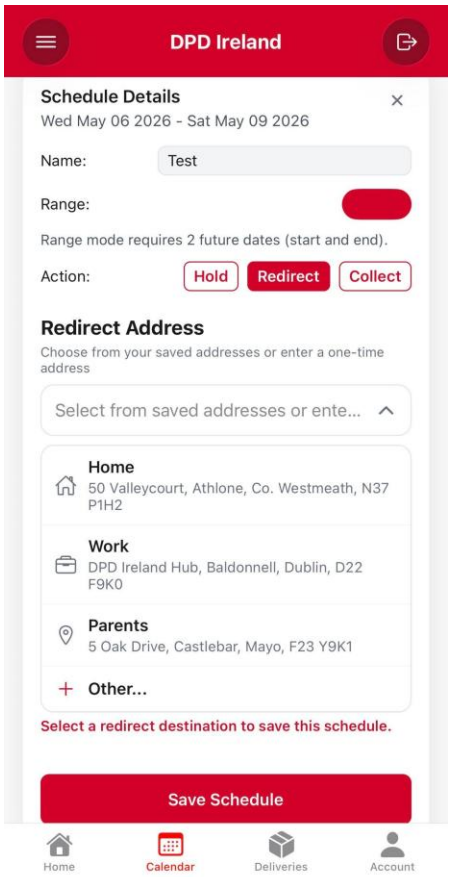
```
{ code: "H", label: "Hold" },
{ code: "A", label: "Redirect" },
{ code: "C", label: "Collect" },
```

These action codes are then used by the calendar through scheduleMap to decide which marker should appear on each date.

Only one action can be selected at a time. The selected action is styled differently using the DPD red colour, making it clear which option is active.

12.4.5.1 Redirect Address Selection

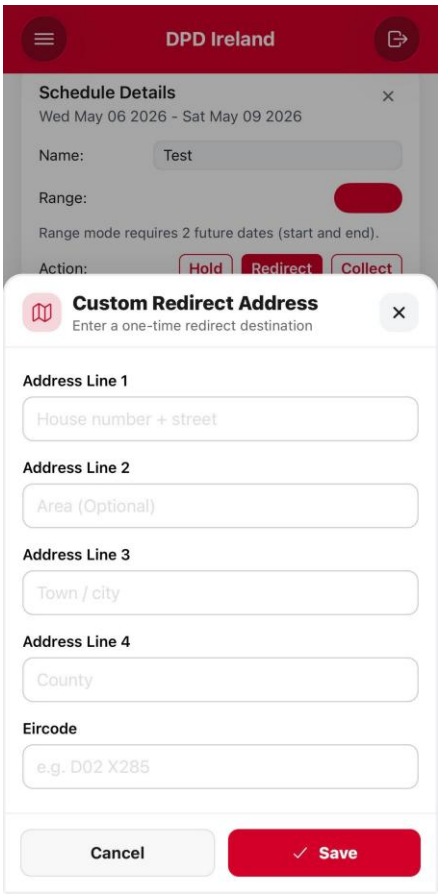
The Redirect action has additional behaviour. If the user selects Redirect, the form displays an address selection area using *AddressBookDropdown.*



This dropdown allows the user to choose from saved addresses in their address book or enter a one-time custom redirect address. It receives the list of addresses from the *useAddresses()* hook.

This is a major UX enhancement as users should not have to repeatedly type an address that is already saved in their account. It also reduces errors because saved address data is more likely to be correct and consistently formatted.

If the user chooses an existing address, the selected address ID is stored. If they choose another address, the screen opens *NewAddressForm* with a custom title and subtitle:



12.4.6 Form Validation

Before a schedule can be saved, *handleSave()* performs validation.

First, it checks that the required date selection exists. Single-day mode requires one selected date. Range mode requires two selected dates. If the selected dates do not match the required count, the function returns without saving.

If the selected action is Redirect, the form also validates the redirect address. It checks that the address has at least line 1, 4 and an Eircode. If any of these are missing, the schedule is not saved.

```
function handleSave() {
  if (
    !startDate ||
    !endDate ||
    orderedDates.length !== requiredSelectionCount
  ) {
    return;
  }

  if (action === "A") {
    const line1 = redirectAddress?.line1?.trim() || "";
    const line4 = redirectAddress?.line4?.trim() || "";
    const eircode = redirectAddress?.eircode?.trim() || "";

    if (!line1 || !line4 || !eircode) {
      return;
    }
  }
```

12.4.7 Schedule Payload

Once validation passes, the form creates a schedule payload. For a single-day schedule, the payload contains one date. For a range schedule, it contains both the start and end date.

```
const payload = {
  name: name || `Schedule ${formatDate(startDate)}`,
  dates: dateStrings,
  startDate: formatDate(startDate),
  endDate: formatDate(endDate),
  mode: scheduleMode,
  action,
};
```

If the action is Redirect, it also includes *redirectAddress*. The object is then passed back to the calendar screen through *onSave(payload).*

Back in CalendarScreen.jsx, the *handleSave()* function calls *addSchedule(schedule)* from the useSchedules() hook and then resets the creation state:

```
function handleSave(schedule) {
  addSchedule(schedule);
  setIsCreatingSchedule(false);
  setSelectedScheduleDates([]);
  setShowScheduleDetails(false);
}
```

This means that after saving, the user is returned to the normal calendar view and the newly created schedule appears as a marker on the calendar.

12.4.8 Schedule Map and Calendar Markers

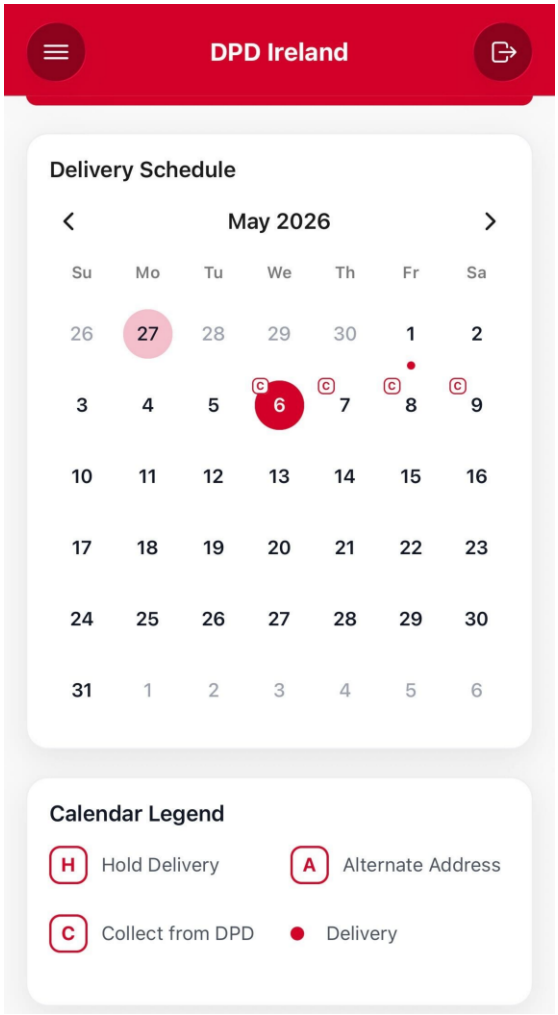
The saved schedules are stored inside *useSchedules.jsx.* This hook keeps an array of schedules and exposes *addSchedule()* and *scheduleMap*.

When a new schedule is added, the hook generates an ID using *Date.now().toString()* and appends it to the schedules array.

The most important derived value is *scheduleMap*. This converts schedules into a Map where the key is a date string and the value is the selected action.

```
// Build a map of date strings to actions for quick lookup in the calendar
const scheduleMap = useMemo(() => {
  const map = new Map();

  schedules.forEach((schedule) => {
    if (schedule.mode === "range") {
      const start = new Date(schedule.startDate);
      const end = new Date(schedule.endDate);

      // Iterate through the selected date range and mark every day.
      for (
        let currentDate = new Date(
          start.getFullYear(),
          start.getMonth(),
          start.getDate(),
        );
        currentDate <= end;
        currentDate.setDate(currentDate.getDate() + 1)
      ) {
        const key = formatDateKey(currentDate);
        map.set(key, schedule.action);
      }
    } else {
      // Add entries for single-day schedules.
      schedule.dates.forEach((date) => {
        map.set(date, schedule.action);
      });
    }
  });

  return map;
}, [schedules]);
```

For single-day schedules, each selected date is added directly to the map. For range schedules, the hook loops from the start date to the end date and marks every date in that range.

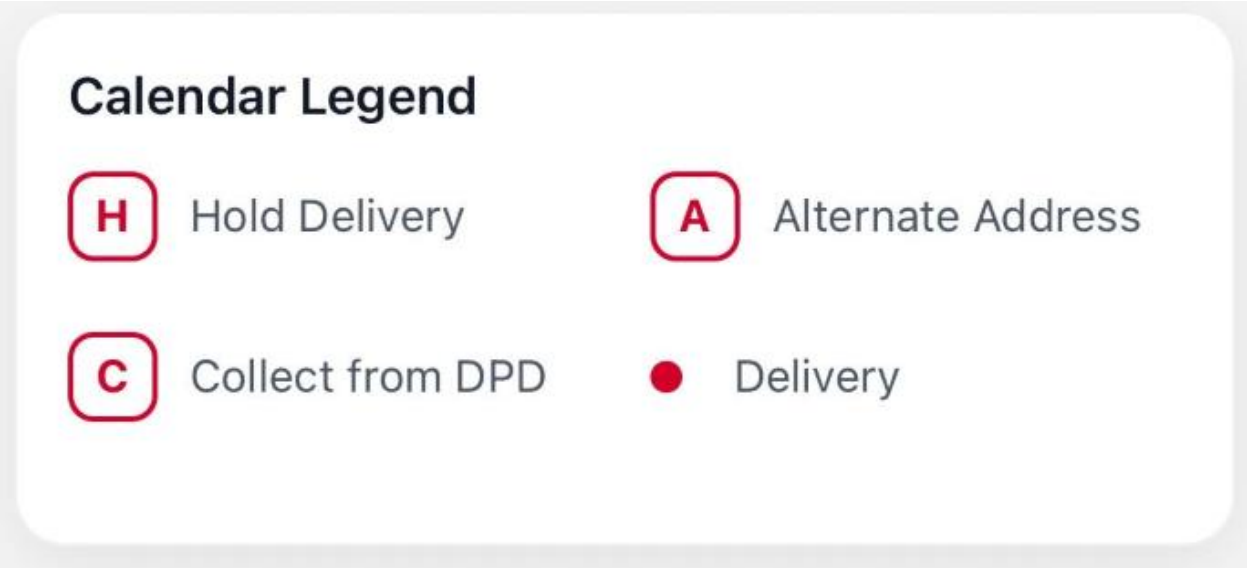


This is important because the calendar cells do not need to understand the full schedule object. They only need to check whether their date key exists in *scheduleMap*. If it does, the cell can show the correct visual marker for that action.

This keeps schedule lookup fast and prevents complex date-range logic from being repeated inside the visual calendar components.

12.5 Calendar Legend

At the bottom of the screen, CalendarLegend.jsx explains the meaning of the symbols used in the calendar:



Each item is rendered using a reusable *LegendItem.jsx*, which either shows a small badge or a delivery dot alongside its label.

A legend is a necessary feature because the calendar contains several visual markers, and without it the meaning of these could be unclear to the user.

12.6 Calendar Summary

The calendar feature is structured again with separation of concerns evident in the design decisions.

The calendar was designed as both a parcel-visibility tool and a delivery-scheduling interface. Its architecture separates date logic, schedule management, and presentation into dedicated hooks and components, allowing the feature to remain modular and scalable. By combining parcel delivery indicators, date selection, and recurring schedule support within one screen, the calendar provides users with both visibility and control over upcoming deliveries.

13 Manage Delivery and Delivery History

This screen is rendered through the tab route *app/(tabs)/*manage, which renders ManageScreen.*jsx.* ManageScreen is divided into two tabs:

- Incoming
- History

Incoming parcels are actionable, while delivery history is primarily informational.

13.1 Main Screen Logic

ManageScreen.jsx controls the overall workflow of the feature. It stores three pieces of local state:

```
const [activeTab, setActiveTab] = useState("incoming");
const [showManageForm, setShowManageForm] = useState(false);
const [selectedParcel, setSelectedParcel] = useState(null);
```

At the same time, it reads parcel lists and update functionality from the *useParcels()* hook.

```
const { inTransit, recently Delivered, updateParcel } = useParcels();
```

13.1.1 Incoming and History List Preparation

The screen builds two memoized arrays:

- incomingParcels, sorted by nearest expected delivery date first
- historyParcels, sorted by most recently delivered first

```
// Show all incoming parcels and sort by soonest delivery date
const incomingParcels = useMemo(() => {
  return [...inTransit].sort(
    (a, b) => new Date(a.expectedAt) - new Date(b.expectedAt),
  );
}, [inTransit]);

// Sort history by most recent delivery date
const historyParcels = useMemo(() => {
  return [...recently Delivered].sort(
    (a, b) => new Date(b.expectedAt) - new Date(a.expectedAt),
  );
}, [recentlyDelivered]);
```

This way the user can see the next incoming parcel and the latest delivered parcel at the top of their respective lists.

13.2 Tab Switching

The screen uses the reusable IncomingHistoryToggle component to switch between the two views.

The toggle itself is implemented as two pressable tab buttons, with the active one given a white background and shadow while the inactive one stays greyed out. The text also shows counts in the labels.

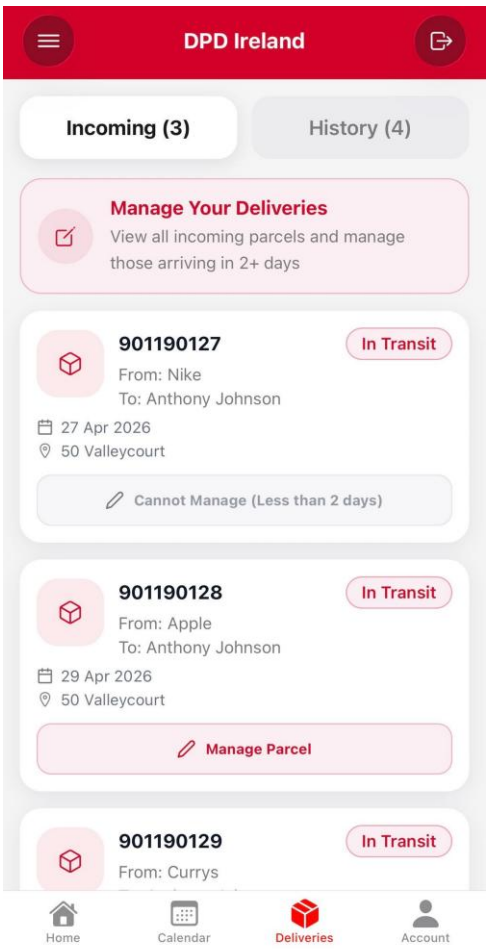
13.3 Incoming Parcels View

When *activeTab === "incoming",* the screen displays the list of sorted incoming parcels using *IncomingParcelCard.*

Before rendering each card, the screen calculates how many days remain until the parcel's expected delivery date. That logic is handled in getDaysUntil(expectedAt) and compares the expected date to today, normalised to midnight to avoid time-of-day issues.

The result is used to decide whether or a parcel can be managed.

This business rule means that parcels arriving too soon are still visible, but the user cannot modify them anymore.



13.3.1 Incoming Parcel Card

The incoming parcel list is rendered using *IncomingParcelCard.jsx*

The top half of the card is pressable and navigates to the parcel details screen, while the bottom button is reserved for managing the delivery.

13.3.2 Manage Availability

As aforementioned, the parcel cannot be managed if the parcel is due to arrive within a day. I use a *canManage* prop to determine the behaviour and styling of the button.

```
<Pressable
  disabled={!canManage}
  onPress={onManagePress}
  style={({ pressed }) => [
    styles.manageButton,
    !canManage && styles.manageButtonDisabled,
    pressed && canManage && styles.pressed,
  ]}
>
```

Also, the label will change to communicate the rule clearly to the user.

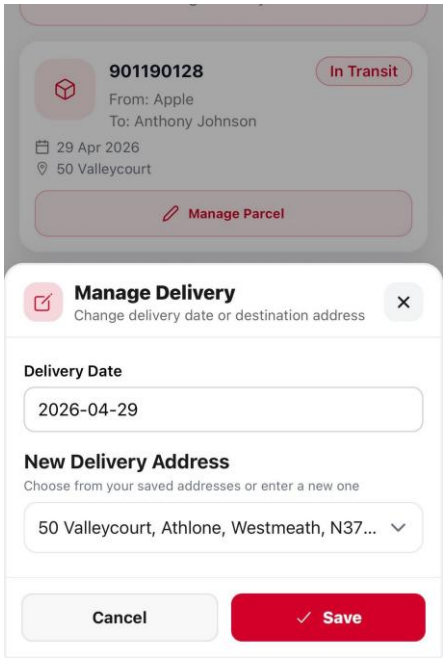
```
<Text
  style={[styles.manageText, !canManage && styles.manageTextDisabled]}
>
  {canManage ? "Manage Parcel" : "Cannot Manage (Less than 2 days)"}
</Text>
```

13.4 Manage Delivery Form

If the user taps Manage Parcel on an eligible parcel, ManageScreen.jsx stores that parcel in selectedParcel and opens ManageDeliveryForm. This form is used when the user wants to change the expected delivery date or redirect the parcel to a different destination.

The form is presented as a bottom-sheet style modal. This keeps the user within the manage delivery screen rather than navigating them away to a separate page

The form is prefilled from the selected parcel. The current expected delivery date is placed into the delivery date input, while the parcel's current address is used as the initial address



13.4.1 Address Selection

Like in the schedule creator, the delivery address is handled through *AddressBookDropdown*. This lets the user either choose one of their saved addresses or enter a one-time custom delivery address.

The form gets saved addresses using the *useAddresses()* hook and then the selected saves address is then derived from *selectedAddressID:*

```
const selectedSavedAddress = useMemo(
  () => addresses.find((address) => address.id === selectedAddressId) || null,
  [addresses, selectedAddressId],
);
```

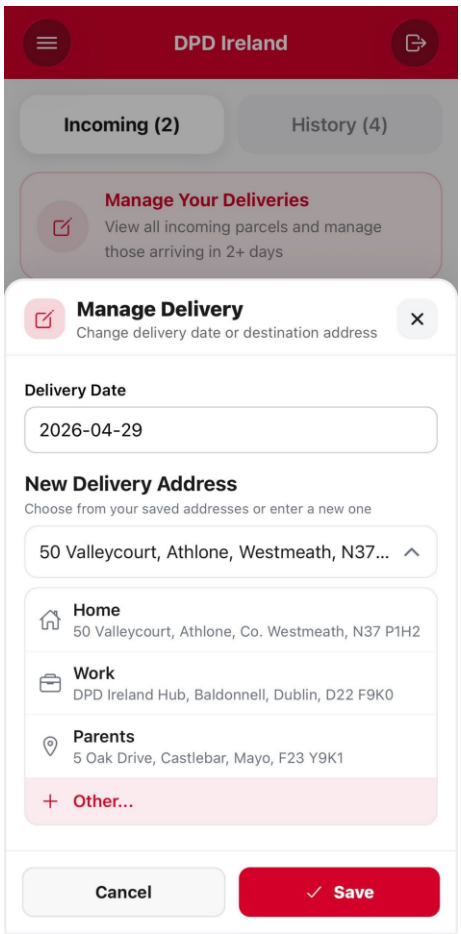
This means the form does not need to copy the full saved address into state immediately. It only stores the selected ID, then derives the matching address from the address book.

If the user chooses a saved address, the form stores the address ID. If the user chooses the custom option, the form clears the saved address selection and opens *NewAddressForm.*

When the custom form is submitted, the entered values are stored in *customAddress,* the saved address selection is cleared, and the custom form closes. This gives the user two clear address paths:

- Choose an existing saved address
- Enter a one-time delivery address

This is better than forcing the user to manually type an address every time they want to redirect a parcel. It also connects the manage delivery flow to the account/address book area of the app.



13.4.2 Validation inside the Form

The form validates the delivery date and address before submitting.

For the date field, the user must enter a date in YYYY-MM-DD format. This is checked using the *isValidDateKey()* helper. The helper first checks the text pattern:

```
function isValidDateKey(dateText) {
  if (!/^\d{4}-\d{2}-\d{2}$/.test(dateText)) {
    return false;
  }
```

It then splits the date into year, month and day, creates a JavaScript Date object, and checks whether the parsed date still matches the original values.

This second step is crucial because a simple pattern check would allow impossible dates. For example, a string could match the YYYY-MM-DD pattern while still not being a real calendar date. By comparing the parsed Date object back against the entered values, the form avoids accepting invalid combinations.

The form also validates that the required address fields exist. It resolves the address by using the selected saved address if one exists, otherwise it falls back to the custom address:

```
const resolvedAddress = selectedSavedAddress || customAddress;
```

Then it trims and extracts the required fields. This validation is intentionally lightweight because this is still a prototype. In a production version, this flow would need more checks, such as preventing past dates, checking public holidays, validating the Eircode against a postal database, and confirming that the new address is serviceable by DPD.

13.4.3 Form Submission Flow

On successful submission, the form sends a compact payload back to *ManageScreen.*

```
onSubmit({
  expectedAt,
  address: {
    line1,
    line2,
    line3,
    line4,
    eircode,
  },
```

Back in ManageScreen, the submitted values are passed into *updateParcel().* The screen chooses the parcel key using either the parcel id or the trackingNumber:

```
function getParcelKey(parcel) {
  return parcel.id || parcel.trackingNumber;
}
```

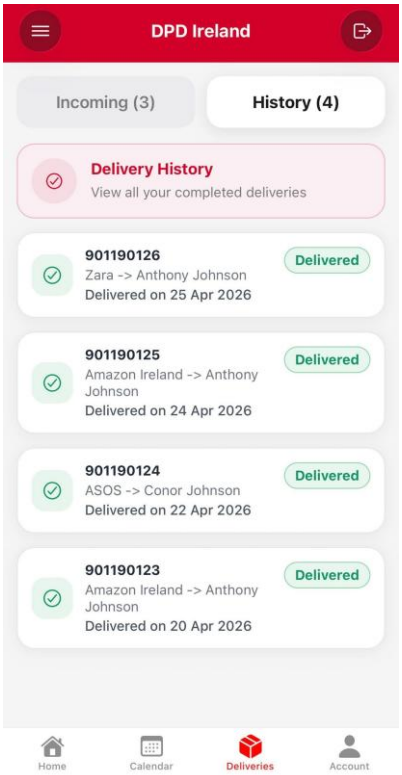
Then the update is applied and the modal is closed.

The form itself is responsible for collecting and validating the new delivery details, while ManageScreen remains responsible for selecting the parcel and applying the update. This separation keeps the feature easier to maintain and follows the same pattern used across the rest of the app.

13.5 History View

When the active tab switches to "history", the same screen changes mode and renders the delivery history list instead of incoming parcels .

This view is intentionally simpler than Incoming because history parcels are no longer actionable. The user can open them to view details, but there is no manage button.



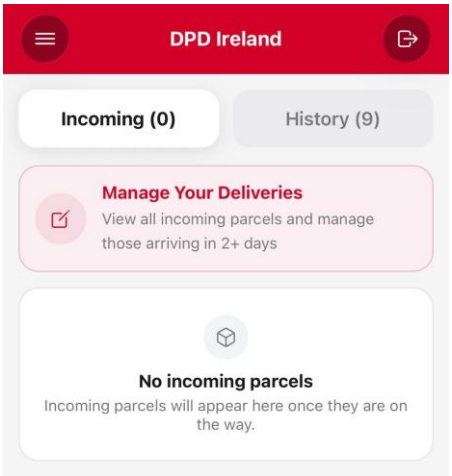
13.5.1 History Parcel Card

Completed deliveries are rendered using *HistoryParcelCard.jsx.* This card is consciously designed to be more compact compared to other cards throughout the project as a users account could potentially contain hundreds of delivered parcels.

13.6 Empty State

Both tabs make use of the same reusable EmptyState component when there are no parcels to show.

- For Incoming, the empty state explains that parcels will appear once they are on the way.
- For History, it explains that delivered parcels will appear once completed.



This is a small but important UX detail because it prevents the screen from feeling broken or unfinished when the lists are empty.

13.7 Manage Delivery and Delivery History Summary

The manage delivery and delivery history feature was implemented as a two-mode screen that separates actionable incoming parcels from completed deliveries. A shared toggle and banner structure provides consistency, while dedicated card components tailor the presentation of each list to its purpose. Incoming parcels can be sorted, checked against delivery-date rules, and updated through a modal management form, whereas history parcels are presented as a

clean read-only record of completed deliveries. This design gives users both control over future deliveries and visibility of past ones within a single organised interface.

14 Utilities and Helpers

A number of small utilities and helper functions are used throughout the project to keep the main screen and component files cleaner. These helpers are not large features by themselves, but they are important because they reduce repetition and keep common logic consistent across the app.

The main helper areas in the project are

- Date utilities
- Parcel status utilities
- Mapping helpers
- Account normalisation
- Schedule lookup helpers
- Local validation helpers

14.1 Date Utilities

The main date utility file is utils/date.ts. This file contains reusable functions used mainly by the calendar and scheduling features.

The app uses dates in several places, including:

- Marking delivery dates on the calendar
- Selecting schedule dates
- Comparing selected dates
- Formatting month labels
- Generating date keys for maps and lookups

Keeping this logic in one utility file avoids duplicating date calculations across multiple components.

14.1.1 Calendar Day Type

```
export type CalendarDay = {
  date: Date;
  day: number;
  inMonth: boolean;
  isToday: boolean;
};
```

This type describes each date cell used in the calendar. Each object contains the actual JavaScript Date, the day number displayed in the UI, whether the date belongs to the current visible month, and whether it is today.

This structure makes the calendar easier to render because DaysGrid and DaysCell receive ready-made date objects rather than calculating this information themselves.

14.1.2 isSameDay()

The *isSameDay()* helper compares two dates by year, month and day:

```
export function isSameDay(a: Date, b: Date): boolean {
  return (
    a.getFullYear() === b.getFullYear() &&
    a.getMonth() === b.getMonth() &&
    a.getDate() === b.getDate()
  );
}
```

This is used when the app needs to check whether two Date objects represent the same calendar day. This is better than directly comparing full Date objects, because two dates can represent the same day but still have different time values.

This helper is also useful in the schedule creation flow, where selected schedule dates need to be added or removed when the user taps on the calendar.

14.1.3 startOfMonth() and addMonths()

The startOfMonth() helper returns the first day of a given month:

```
export function startOfMonth(date: Date): Date {
  return new Date(date.getFullYear(), date.getMonth(), 1);
}
```

This is used by the calendar screen hook to initialise the visible month.

The addMonths() helper returns a new date offset by a number of months:

```
export function addMonths(date: Date, delta: number): Date {
  return new Date(date.getFullYear(), date.getMonth() + delta, 1);
}
```

This is used for month navigation. When the user presses the next or previous month controls, the hook can move the visible month forward or backward.

14.1.4 formatDateKey()

One of the most important date helpers is formatDateKey():

```
export function formatDateKey(date: Date): string {
  const year = date.getFullYear();
  const month = String(date.getMonth() + 1).padStart(2, "0");
  const day = String(date.getDate()).padStart(2, "0");

  return `${year}-${month}-${day}`;
}
```

This converts a Date object into a consistent YYYY-MM-DD key.

This is used throughout the calendar and scheduling logic because date keys are needed for fast comparisons. For example, delivery dates and schedule dates can be stored in a Set or Map, and calendar cells can then check whether their date key exists.

This helper is important because it avoids relying on toISOString().split("T")[0]. I found out the hard way that that approach can introduce timezone issues, because ISO strings are based on UTC. By manually building the date key from the local year, month and day, the app keeps the date aligned with what the user sees on screen.

14.1.5 buildCalendarDays()

The largest helper in date.ts is *buildCalendarDays()*

```
export function buildCalendarDays(monthDate: Date, today: Date): CalendarDay[] {
  const year = monthDate.getFullYear();
  const month = monthDate.getMonth();
  const firstDayOfMonth = new Date(year, month, 1);
  const firstWeekday = firstDayOfMonth.getDay();
  const daysInMonth = new Date(year, month + 1, 0).getDate();
  const daysInPrevMonth = new Date(year, month, 0).getDate();
  const days: CalendarDay[] = [];

  for (let i = firstWeekday - 1; i >= 0; i -= 1) {
    const day = daysInPrevMonth - i;
    const date = new Date(year, month - 1, day);
    days.push({
      date,
      day,
      inMonth: false,
      isToday: isSameDay(date, today),
    });
  }

  for (let day = 1; day <= daysInMonth; day += 1) {
    const date = new Date(year, month, day);
    days.push({
      date,
      day,
      inMonth: true,
      isToday: isSameDay(date, today),
    });
  }

  let nextMonthDay = 1;
  while (days.length % 7 !== 0) {
    const date = new Date(year, month + 1, nextMonthDay);
    days.push({
      date,
      day: nextMonthDay,
      inMonth: false,
      isToday: isSameDay(date, today),
    });
    nextMonthDay += 1;
  }

  return days;
}
```

This function builds the full list of calendar cells for a visible month. It does this by:

- Finding the first day of the month
- Calculating the weekday the month starts on
- Adding trailing days from the previous month
- Adding every day in the current month
- Adding leading days from the next month until the grid is complete

The function returns an array of CalendarDay objects. Each object tells the UI whether the date belongs to the visible month and whether it is today.

This is the function that allows the calendar UI to show a complete month grid rather than only rendering the numbered days of the month. Without this helper, the calendar screen would need to manually calculate blank cells or handle incomplete rows.

14.2 Parcel Status Utilities

Parcel status helpers are stored in utils/parcels.js. These functions are used to turn a parcel's expected delivery date into a meaningful status for the UI. Parcel status in this project is based on the date. If it is due today it is "Out for delivery". If it's a past date it is "Delivered". And if its due for a future date, it is "In Transit"

14.2.1 getParcelStatus()

The getParcelStatus() function compares a parcel's expected delivery date with the current date.

If no expected date is provided, the function defaults the parcel to "IN_TRANSIT"

The function then normalises both today's date and the parcel's expected date to remove the time portion. This is done by creating new dates using only the year, month and day.

14.2.2 getParcelStatusDisplay()

The *getParcelStatusDisplay()* helper converts internal status codes into user-facing labels.

```
export function getParcelStatusDisplay(status) {
  switch (status) {
    case "DELIVERED":
      return "Delivered";
    case "OUT_FOR_DELIVERY":
      return "Out for Delivery";
    case "IN_TRANSIT":
    default:
      return "In Transit";
  }
}
```

This separation is useful because the app can use stable internal values for logic, while still showing readable text in the UI. If the wording ever needs to change, it can be changed in one place without affecting the underlying status logic.

These parcel helpers are mainly used by ParcelContext, where raw parcel data is enriched with both status and statusDisplay. Screens and cards can then simply read those values instead of calculating them themselves.

14.3 Mapping Helpers

The project also uses mapping helpers to convert backend data into the frontend parcel model.

The main mapper files are:

- mapConsignment.js
- mapConsignments.js

These helpers are important because the backend consignment shape did not exactly match the shape used by the frontend components. This will more than likely be the case when integrating with a larger backend in the future, so having these helpers here will reduce friction in the future.

14.3.1 mapConsignmentToParcel()

mapConsignmentToParcel() converts a single backend consignment into the parcel object used in the app. It also creates a nested address object:

```
export default function mapConsignmentToParcel(consignment) {
  return {
    id: String(consignment.consignment_number),
    trackingNumber: String(consignment.consignment_number),

    toName: consignment.name,
    fromName: consignment.sender_name,

    address: {
      line1: consignment.addressline1,
      line2: consignment.addressline2 || "",
      line3: consignment.addressline3,
      line4: consignment.addressline4,
      eircode: consignment.eircode,
    },

    eta: {
      label: formatDateLabel(consignment.expected_delivery_date),
    },

    shippedAt: consignment.shipped_at,
    expectedAt: consignment.expected_delivery_date,
    status: consignment.status,
    statusDisplay: consignment.statusDisplay,

    deliveryDepot: consignment.delivery_depot,
    weight: consignment.weight,
    accountNo: consignment.account_no,
  };
}
```

14.3.2 mapConsignmentsToParcel()

mapConsignmentsToParcels() is a simple wrapper that maps an array of consignments through *mapConsignmentToParcel():*

```
export default function mapConsignmentsToParcels(consignments = []) {
  return consignments.map(mapConsignmentToParcel);
}
```

This allows the fetching of all consignments belonging to an account and then mapping them to the frontend shape.

These mappers give the app one central place to change how backend consignments are transformed in future.

15 Ethics

As the project is a customer-facing logistics application, ethical considerations around privacy, security, accessibility, and responsible data handling are important. A Parcel Wizard application would handle personal information such as names, addresses, phone numbers, email addresses, account numbers, parcel tracking numbers, and delivery preferences. This data must be treated carefully because it could reveal where a person lives, when they are likely to be away from home, and what parcels they are receiving.

For this reason, the integration of a DPD backend environment was not possible. The prototype was developed using mock data and a mock DPD-style backend so that the app could be tested safely without exposing personal customer information.

Security is also handled in the app. Login credentials and account tokens are stored using secure storage rather than ordinary local storage. The app also separates authentication, account data, and parcel data into controlled context layers, which helps avoid unnecessary exposure of sensitive information across the application.

16 Conclusion

The outcome of this project is a working mobile prototype of a modern DPD Parcel Wizard application. The app demonstrates the main features required for a customer-facing parcel management system.

The project successfully brought together research, design, frontend development, backend integration, and user experience considerations into a demonstrable system. React Native and Expo were used to build a cross-platform mobile application, while a mock DPD-style backend was used to support realistic account and parcel data during development. The use of service layers, context providers, mappers, and normalisers also means the application has been structured with future backend integration in mind.

Overall, the project achieved its main goal of redesigning Parcel Wizard as a cleaner, more modern, and more user-friendly mobile experience. The final prototype is being presented to DPD next week, where feedback will be gathered on how well it aligns with the needs of the business and its customers. If the response is positive, hopefully you'll be managing your parcels with Parcel Wizard in the near future.

17 Appendix

17.1 Github Link

Main code repository can be found here - https://github.com/Antoj2000/DPDParcelWizard

17.2 Short Demo Video

A 2:40 second video summary here - https://youtu.be/fOWaHsNhDfo

17.3 Longer Demo Video

A longer video mp4 file detailing all features available to download below

PWdemo.mp4

18 References

[1] DPD Parcel Wizard, "DPD Parcel Wizard Website," DPD, [Online]. Available: https://www.dpdparcelwizard.ie/HomePage#/.

[2] DPD Ireland, “DPD Ireland App Google Play Store,” DPD Ireland, [Online]. Available: https://play.google.com/store/apps/details?id=ie.dpd.parcelwizard&hl=en_IE.

[3] Expo, "Expo Router Docs," Expo, [Online]. Available: https://docs.expo.dev/router/introduction/.

[4] React Navigation, "React Navigation Drawer," [Online]. Available: https://reactnavigation.org/docs/drawer-navigator/.

[5] React Navigation, "React Navigation Tabs," [Online]. Available: https://reactnavigation.org/docs/tab-view/.

[6] React, "React Context," [Online]. Available: https://react.dev/reference/react/useContext.

[7] OpenAI, "ChatGPT," OpenAI, [Online]. Available: https://chatgpt.com/.

[8] Expo, "Expo Secure-Store Docs," [Online]. Available: https://docs.expo.dev/versions/latest/sdk/securestore/.

[9] React, "React Animations," [Online]. Available: https://reactnative.dev/docs/animations.

[10] Expo, "Expo-Camera Docs," Expo, [Online]. Available: https://docs.expo.dev/versions/latest/sdk/camera/.

[11] Expo, "Expo Location Docs," [Online]. Available: https://docs.expo.dev/versions/latest/sdk/location/.

[12] Expo, "Expo Maps Docs," [Online]. Available: https://docs.expo.dev/versions/latest/sdk/maps/.

[13] Ionic, "Ionicons Library," [Online]. Available: https://ionic.io/ionicons.